
iscte
INSTITUTO
UNIVERSITÁRIO
DE LISBOA
emprego
digital
UPskill

What is a Database?

Organized collection of data stored in an electronic format

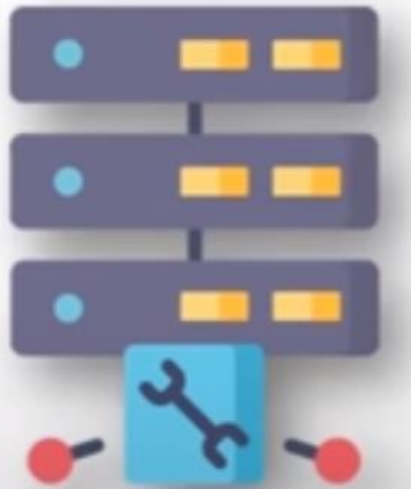
Data

Database

- Access Data
- Manipulate Data
- Update Data

Database Management System

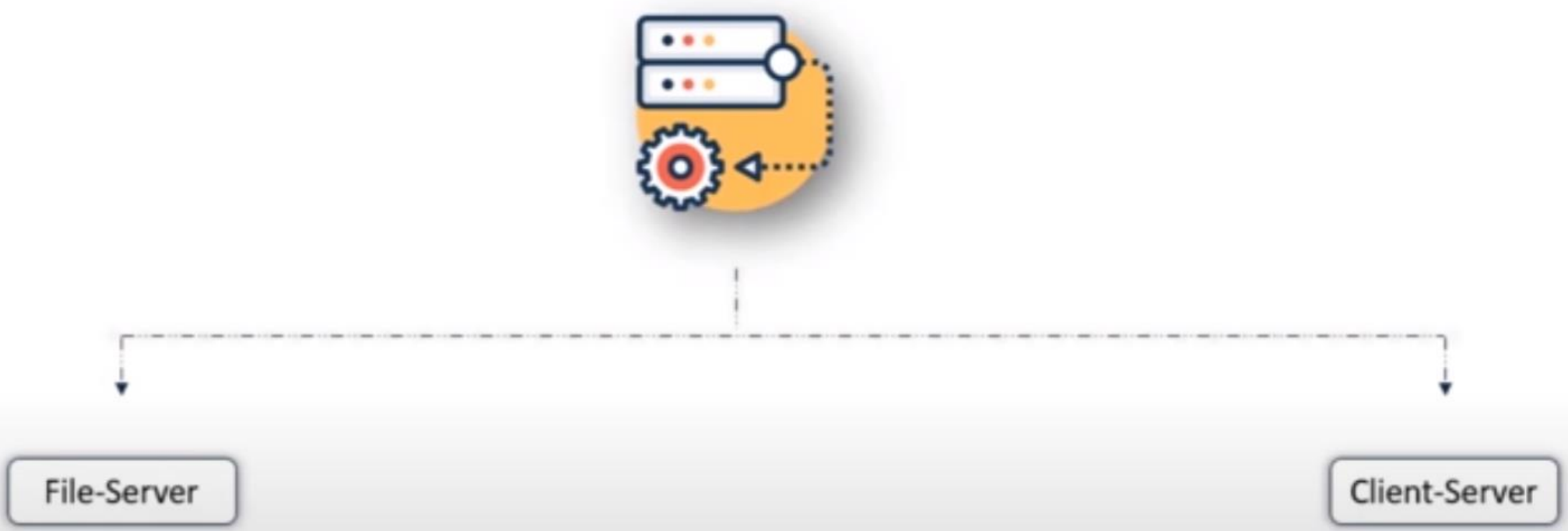
DBMS is a system software for creating and managing databases



Database

DBMS

Types of Database Architecture



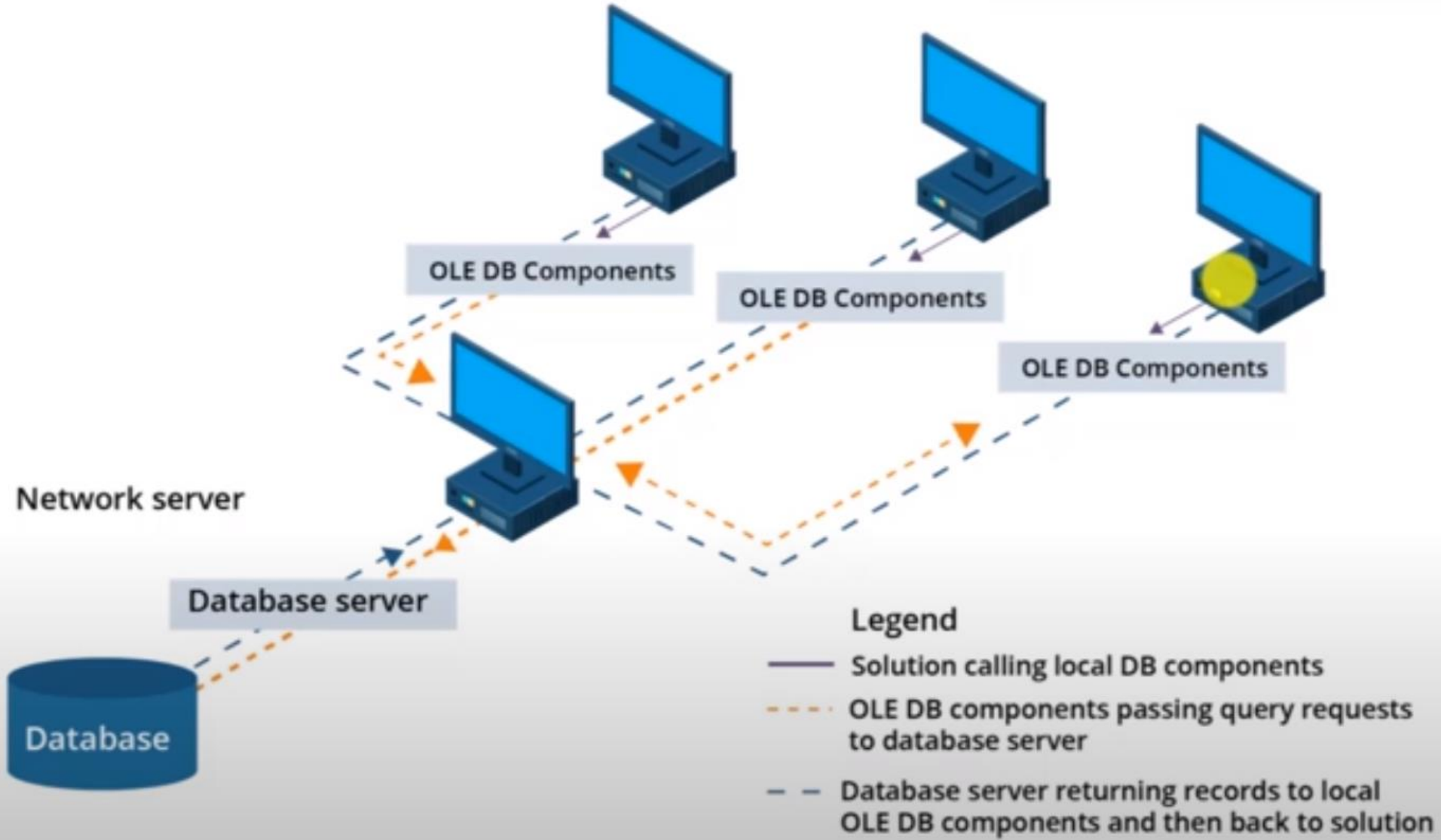
File Server Architecture

Files

Local System

File Server Architecture

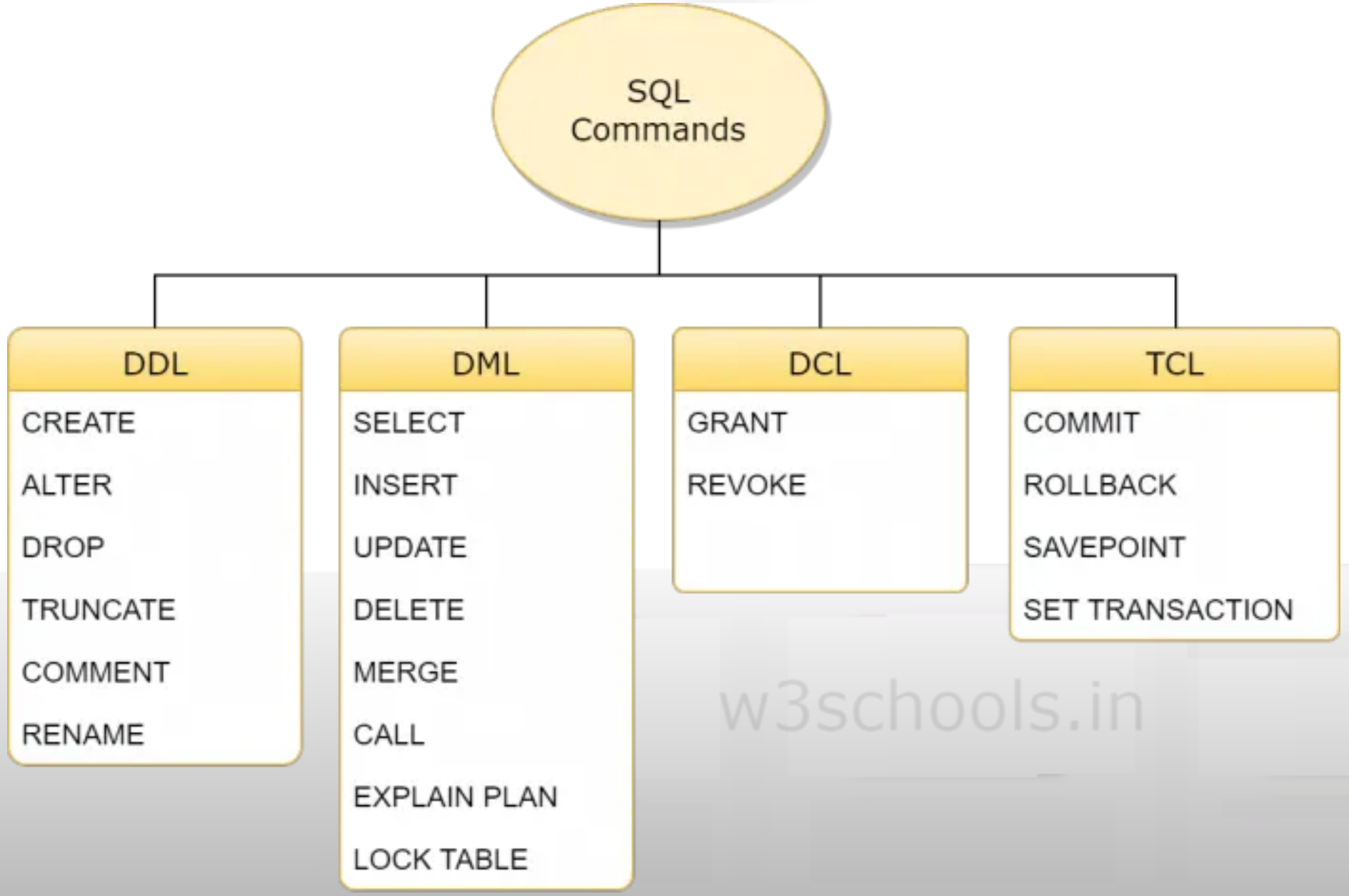
Client Server Architecture



Introduction to SQL

SQL stands for Structured Query Language which is a standard language for accessing & manipulating databases

Categories of SQL Commands



Tables in SQL

A Table is a database object which comprises of rows and columns

e_id	e_name	e_salary	e_age	e_gender	e_dept
1	Sam	95000	45	Male	Operations
2	Bob	80000	21	Male	Support
3	Anne	125000	25	Female	Analytics
4	Julia	73000	30	Female	Analytics
5	Matt	159000	33	Male	Sales
6	Jeff	112000	27	Male	Operations

Fields & Records

e_id	e_name	e_salary	e_age	e_gender	e_dept
1	Sam	95000	45	Male	Operations
2	Bob	80000	21	Male	Support
3	Anne	125000	25	Female	Analytics
4	Julia	73000	30	Female	Analytics
5	Matt	159000	33	Male	Sales
6	Jeff	112000	27	Male	Operations

Record

e_id	e_name	e_salary	e_age	e_gender	e_dept
1	Sam	95000	45	Male	Operations
2	Bob	80000	21	Male	Support
3	Anne	125000	25	Female	Analytics
4	Julia	73000	30	Female	Analytics
5	Matt	159000	33	Male	Sales
6	Jeff	112000	27	Male	Operations

Field

Fields

A Field provides specific information about data in the table

e_id	e_name	e_salary	e_age	e_gender	e_dept
1	Sam	95000	45	Male	Operations
2	Bob	80000	21	Male	Support
3	Anne	125000	25	Female	Analytics
4	Julia	73000	30	Female	Analytics
5	Matt	159000	33	Male	Sales
6	Jeff	112000	27	Male	Operations

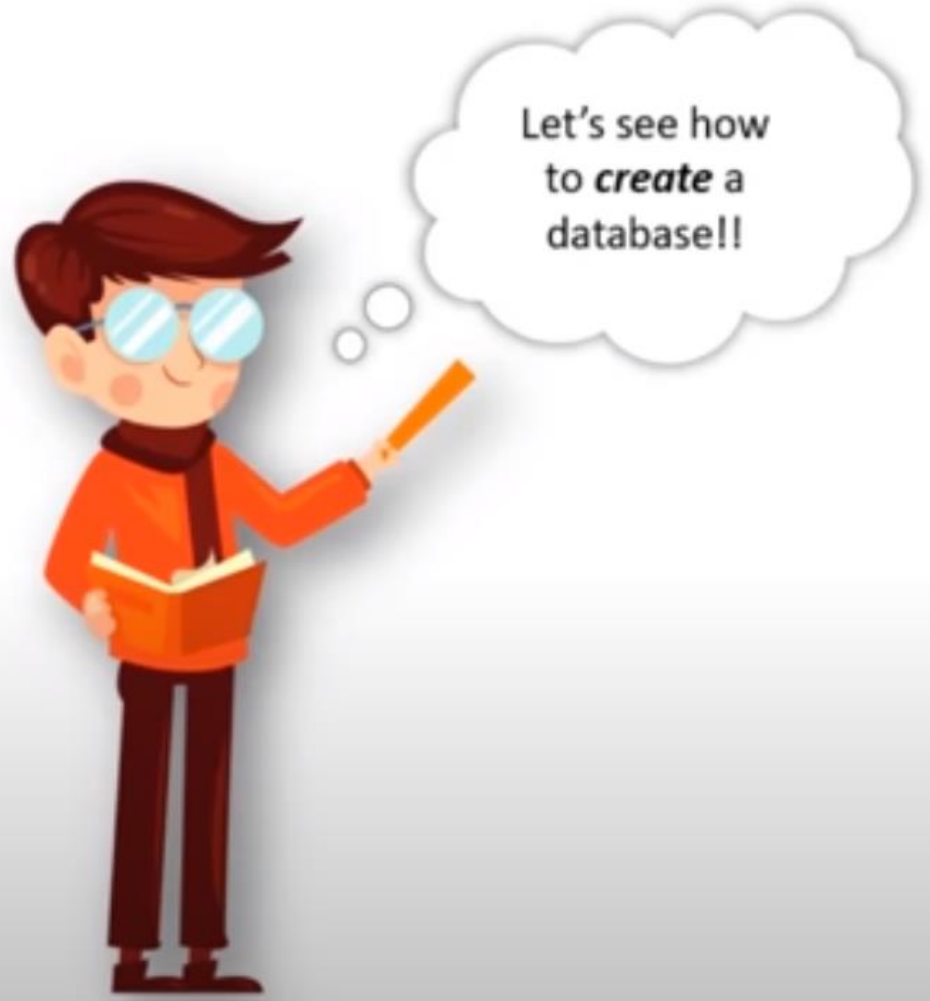
Records

Each individual entry of a table is called the record

e_id	e_name	e_salary	e_age	e_gender	e_dept
1	Sam	95000	45	Male	Operations
2	Bob	80000	21	Male	Support
3	Anne	125000	25	Female	Analytics
4	Julia	73000	30	Female	Analytics
5	Matt	159000	33	Male	Sales
6	Jeff	112000	27	Male	Operations

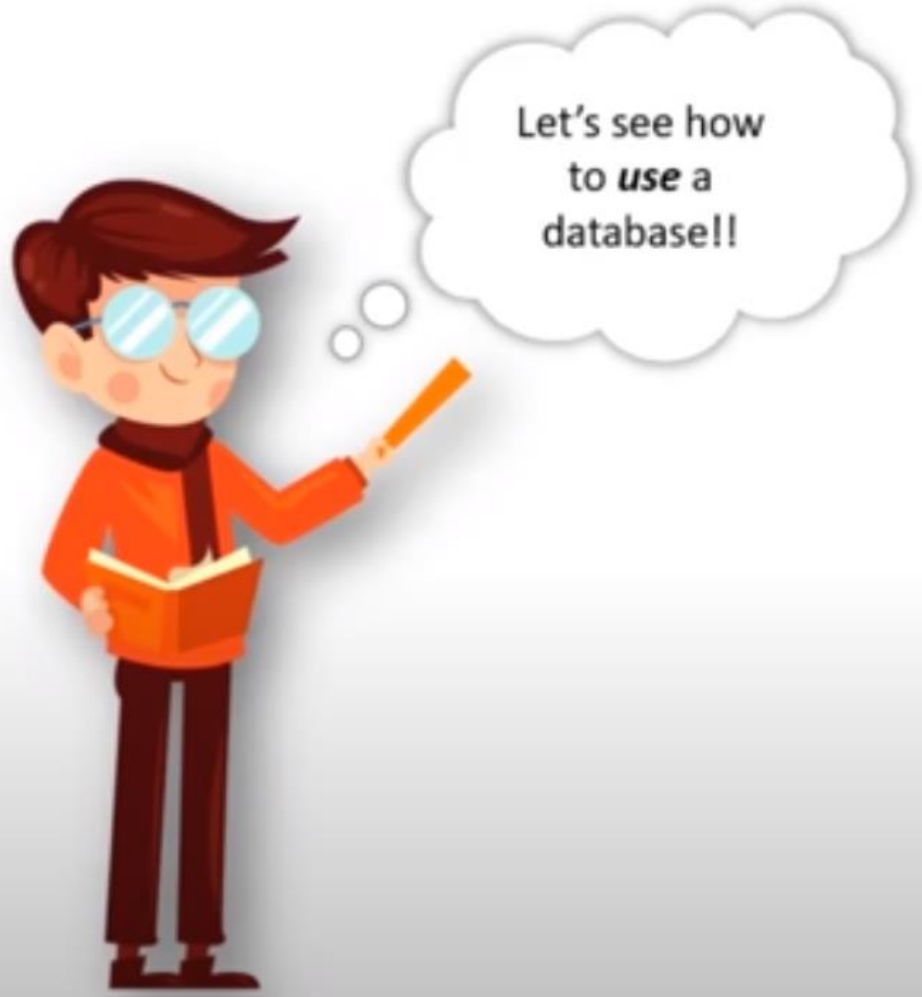
Create, Use, and Drop Database in SQL

Creating A Database: Syntax



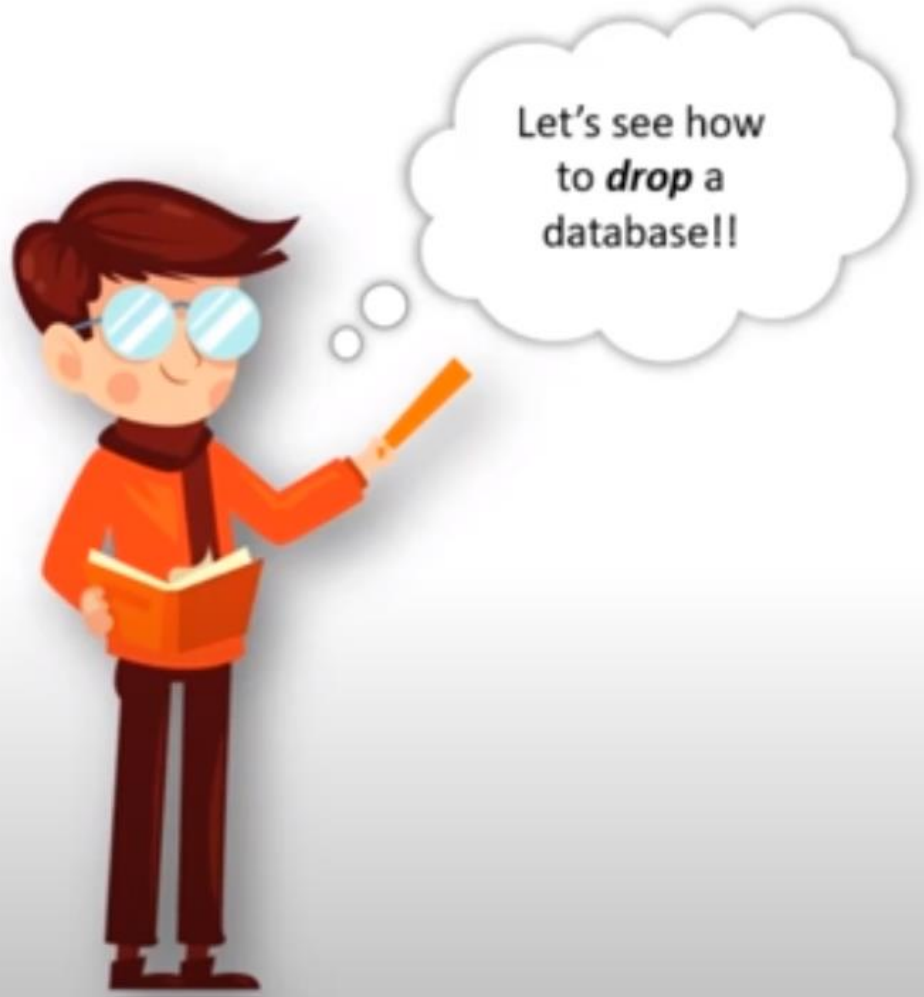

```
CREATE DATABASE databasename;
```

Use A Database: Syntax




```
USE [DatabaseName];
```

Drop A Database: Syntax




```
DROP DATABASE databasename;
```

Data Types in SQL

Data Types define what type of data a column can hold

e_id	e_name	e_salary	e_age	e_gender	e_dept
1	Sam	95000	45	Male	Operations
2	Bob	80000	21	Male	Suppor
3	Anne	125000	25	Female	Analytics
4	Julia	73000	30	Female	Analytics
5	Matt	159000	33	Male	Sales
6	Jeff	112000	27	Male	Operations

Integer

Character

Different Data Types in SQL

Numerical Data Types

Data Type	Range
bigint	-9,223,372,036,854,775,808 <-> 9,223,372,036,854,775,807
int	-2,147,483,648 <-> 2,147,483,647
smallint	-32,768 <-> 32,767
tinyint	0 <-> 255
decimal(s,d)	-10^38 +1 <-> 10^38 -1

Character Data Types

Data Type	Range
char(s)	255 characters
varchar(s)	255 characters
text	65,535 characters

Date & Time Data Types

Data Type	Format
date	YYYY-MM-DD
time	HH:MM:SS
Year	YYYY

Constraints in SQL

Constraints are used to specify rules for data in table

- Not Null
- Default
- Unique
- Primary Key

Not Null Constraint

Not Null Constraint ensures that a column cannot have a NULL value

e_id	e_name	e_salary	e_age	e_gender	e_dept
1	Sam	95000	45	Male	Operations
2	Bob	80000	21	Male	Support
3	Anne	125000	25	Female	Analytics
4	Julia	73000	30	Female	Analytics
5	Matt	159000	33	Male	Sales
6	Jeff	112000	27	Male	Operations

Default Constraint

Default Constraint sets a default value for a column when no value is specied

e_id	e_name	e_salary	e_age	e_gender	e_dept
1	Sam	123000	25	Male	Operations
2	Bob	75000	25	Male	Support
3	Anne	89000	25	Female	Analytics

Unique Constraint

Unique constraint ensures that all values in a column are different

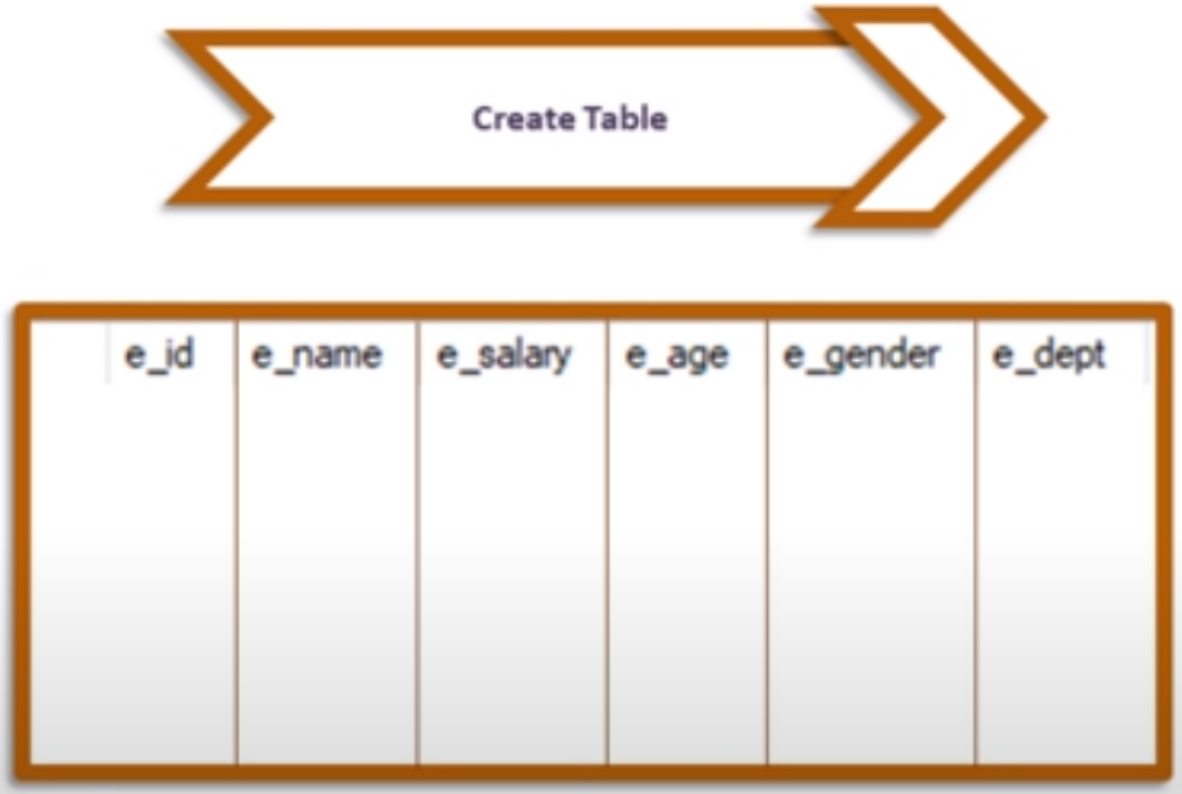
e_id	e_name	e_salary	e_age	e_gender	e_dept
1	Sam	95000	45	Male	Operations
2	Bob	80000	21	Male	Support
3	Anne	125000	25	Female	Analytics
4	Julia	73000	30	Female	Analytics
5	Matt	159000	33	Male	Sales
6	Jeff	112000	27	Male	Operations

Primary Key Constraint

Primary Key constraint uniquely identifies each record in a table

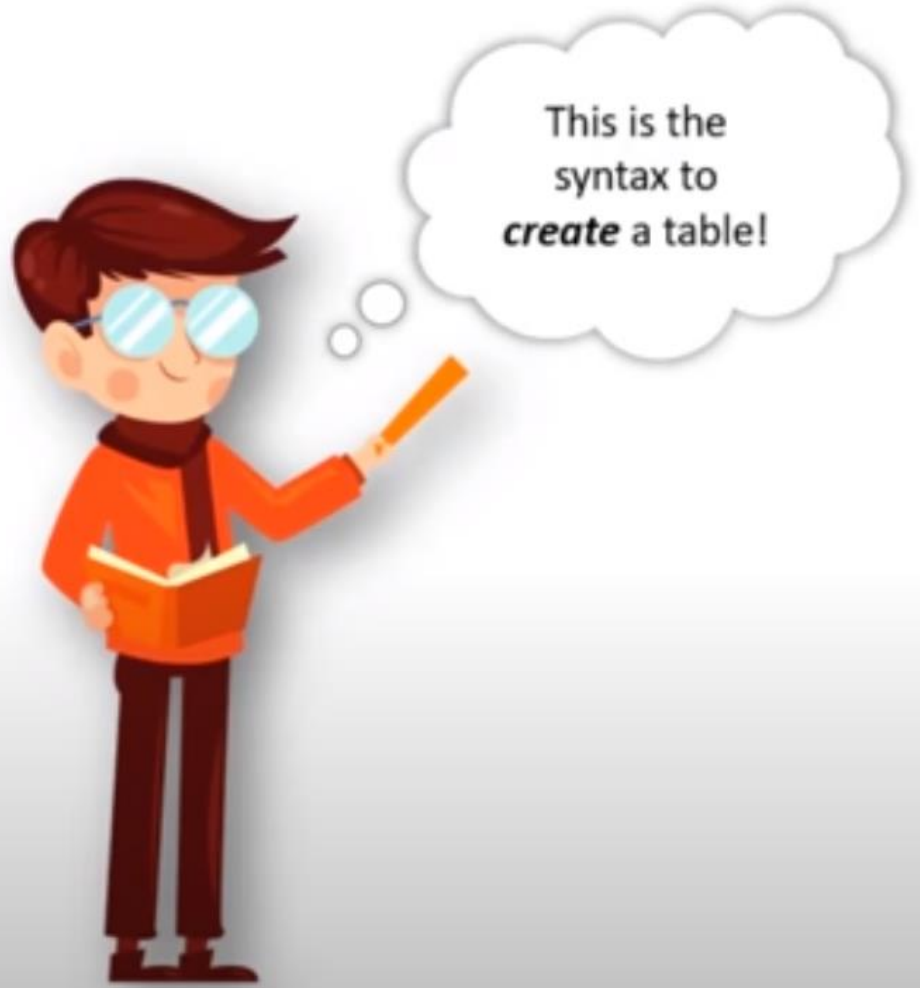
Not Null+Unique

e_id	e_name	e_salary	e_age	e_gender	e_dept
1	Sam	95000	45	Male	Operations
2	Bob	80000	21	Male	Support
3	Anne	125000	25	Female	Analytics
4	Julia	73000	30	Female	Analytics
5	Matt	159000	33	Male	Sales
6	Jeff	112000	27	Male	Operations


Create Table
e_id
e_name
e_salary
e_age
e_gender
e_dept

Create Table

Syntax of Create Table Statement




```
CREATE TABLE table_name(
column1 datatype,
column2 datatype,
column3 datatype,
.....
columnN datatype,
PRIMARY KEY(column_x) );
```

Insert Query Syntax


```
INSERT INTO table_name
VALUES (value1, value2,value3,...valueN);
```

Select Statement Syntax


```
SELECT column1, column2, columnN
FROM table_name;
```

Select Distinct Syntax


```
SELECT DISTINCT column1, column2,
columnN FROM table_name;
```

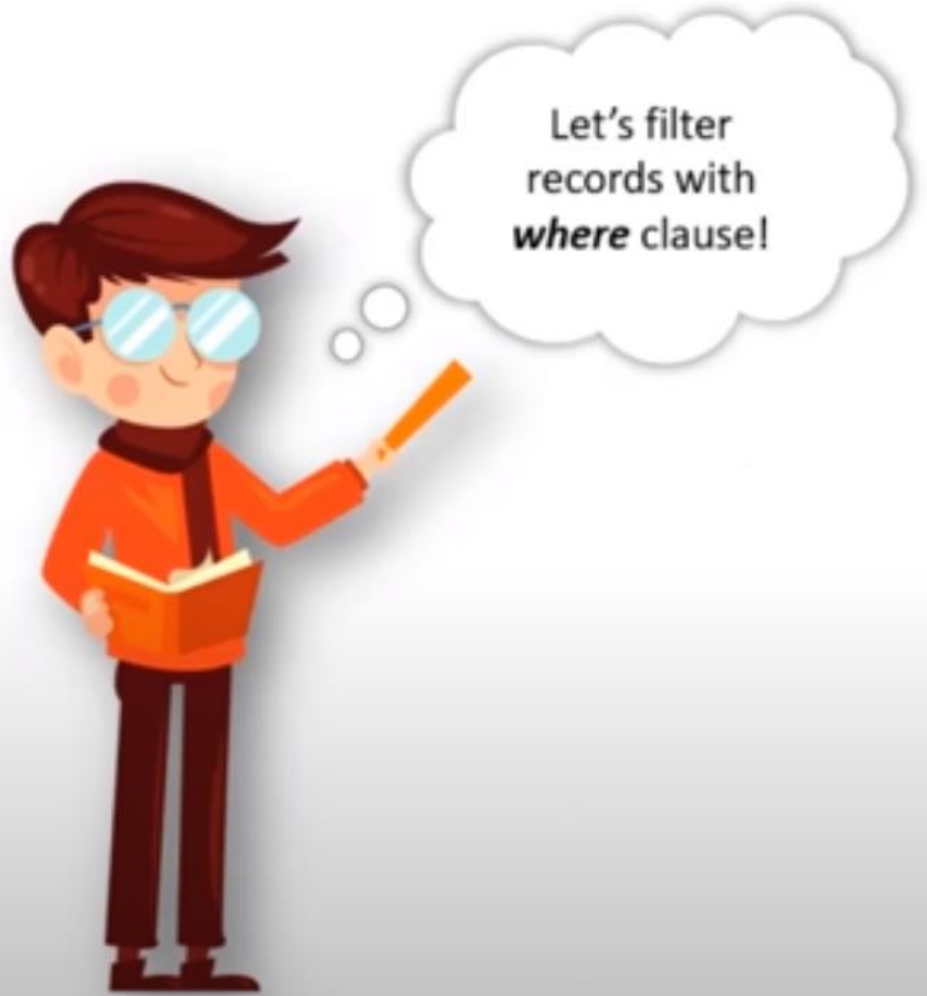
Where Clause

Where Clause is used to extract records which satisfy a condition

Age>60

Occupation="Doctor"

Where Clause Syntax




```
SELECT column1, column2, columnN
FROM table_name WHERE [condition]
```

1. AND Operator
2. OR Operator
3. NOT Operator

e_id	e_name	e_salary	e_age	e_gender	e_dept
1	Sam	95000	45	Male	Operations
2	Bob	80000	21	Male	Support
3	Anne	125000	25	Female	Analytics
4	Julia	73000	30	Female	Analytics
5	Matt	159000	33	Male	Sales
6	Jeff	112000	27	Male	Operations

AND Operator Syntax


```
SELECT column1, column2, columnN
FROM table_name WHERE [condition1]
AND [condition2]...AND [conditionN];
```

OR Operator Syntax


```
SELECT column1, column2, columnN
FROM table_name WHERE [condition1]
OR [condition2]...OR [conditionN];
```

NOT Operator

NOT operator displays a record if the condition is NOT TRUE

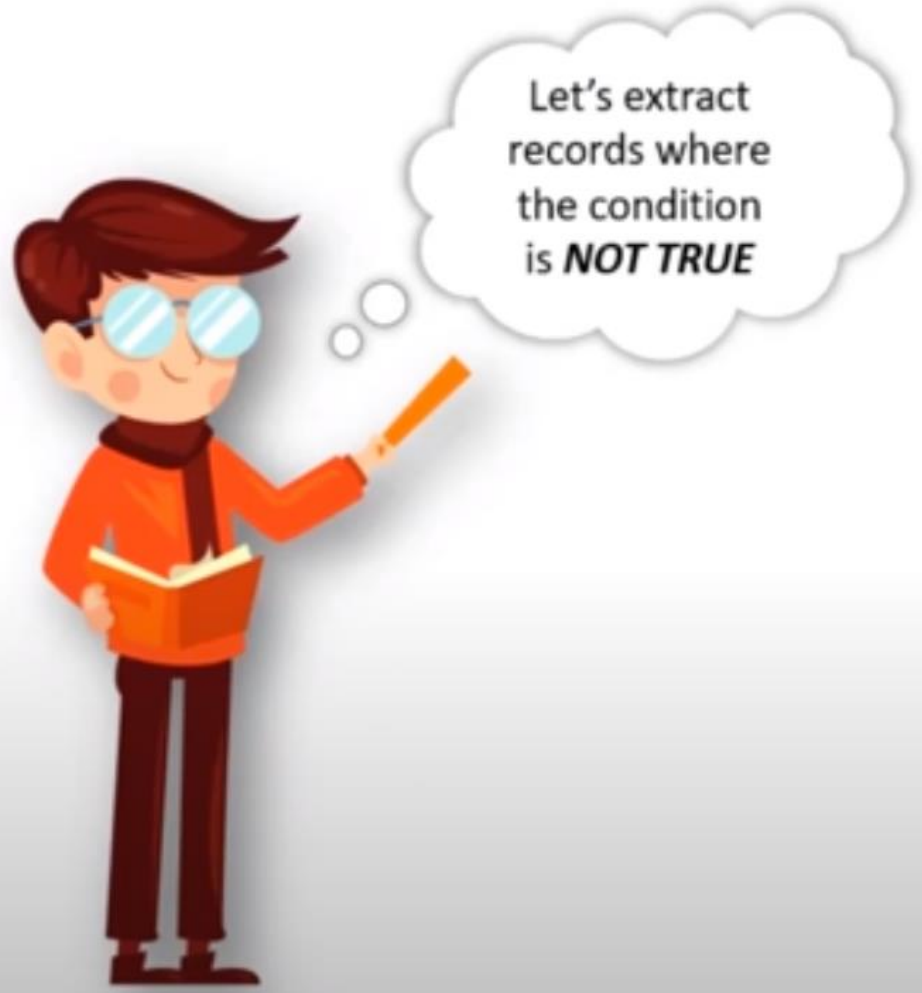
NOT

Occupation="Software Engineer"

Occupation="Doctor"

Occupation="Actor"

NOT Operator Syntax



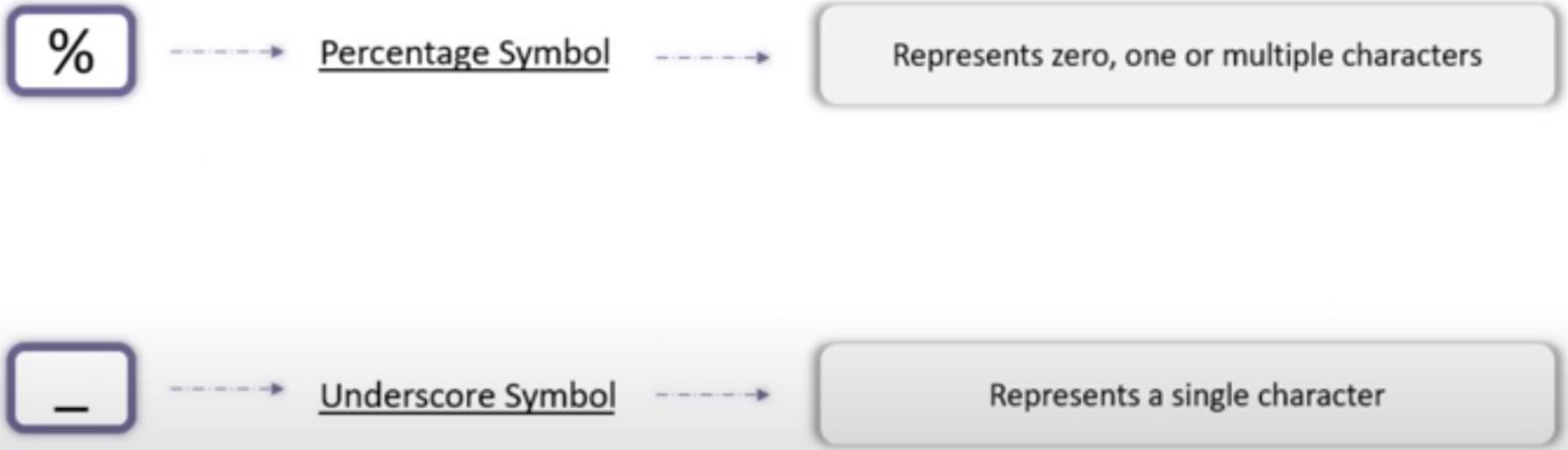

```
SELECT column1, column2, columnN
FROM table_name WHERE NOT
[condition];
```

1. LIKE Operator

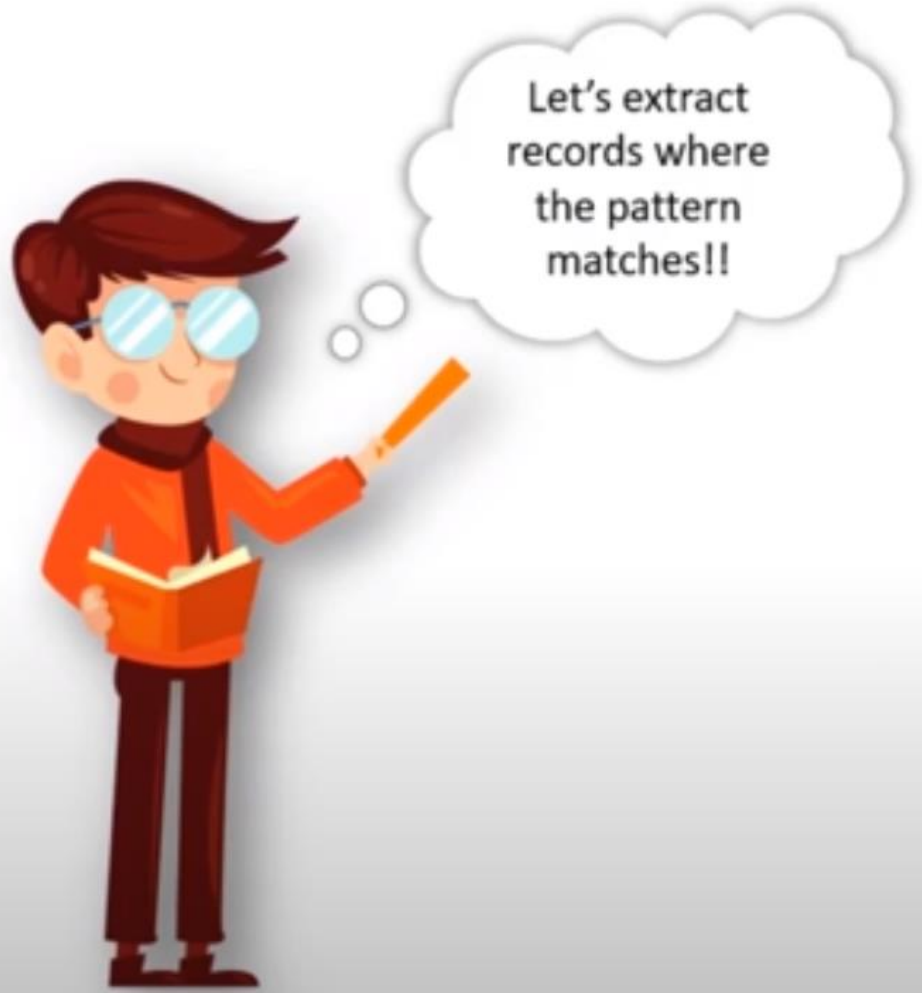
2. BETWEEN Operator

e_id	e_name	e_salary	e_age	e_gender	e_dept
1	Sam	95000	45	Male	Operations
2	Bob	80000	21	Male	Support
3	Anne	125000	25	Female	Analytics
4	Julia	73000	30	Female	Analytics
5	Matt	159000	33	Male	Sales
6	Jeff	112000	27	Male	Operations

Wild Card Characters

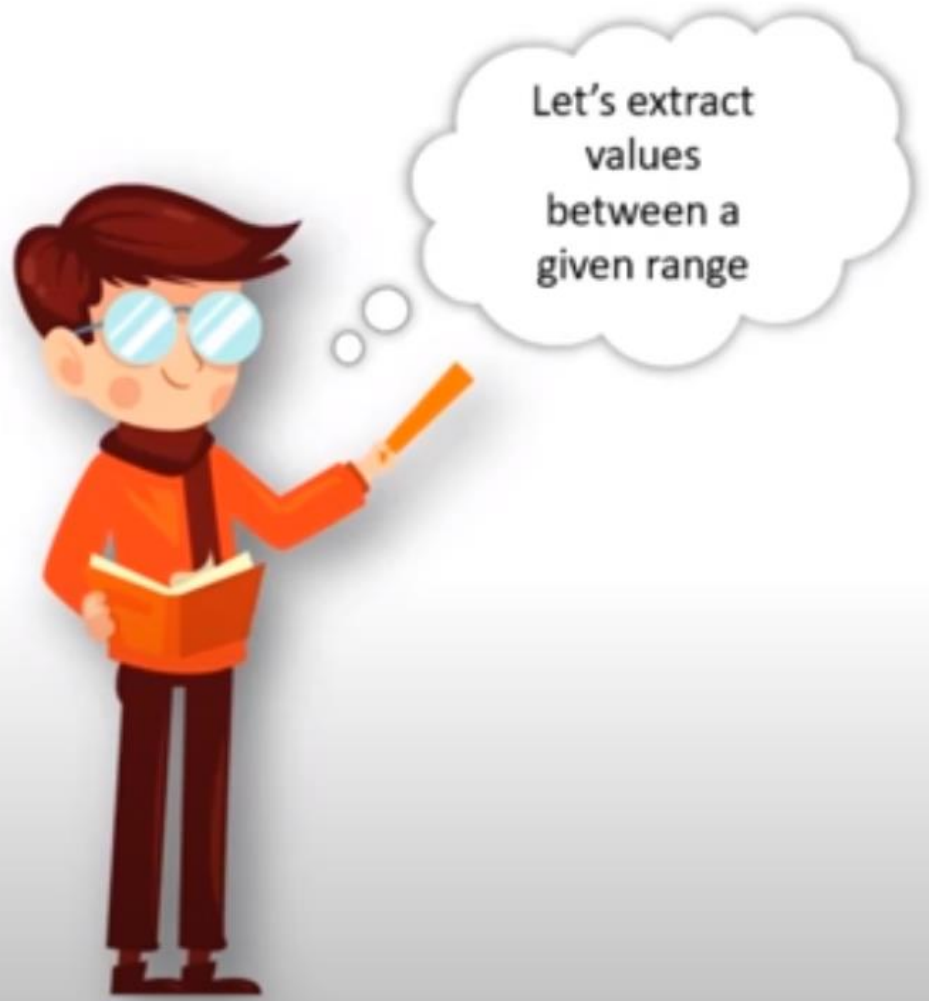


LIKE Operator Syntax

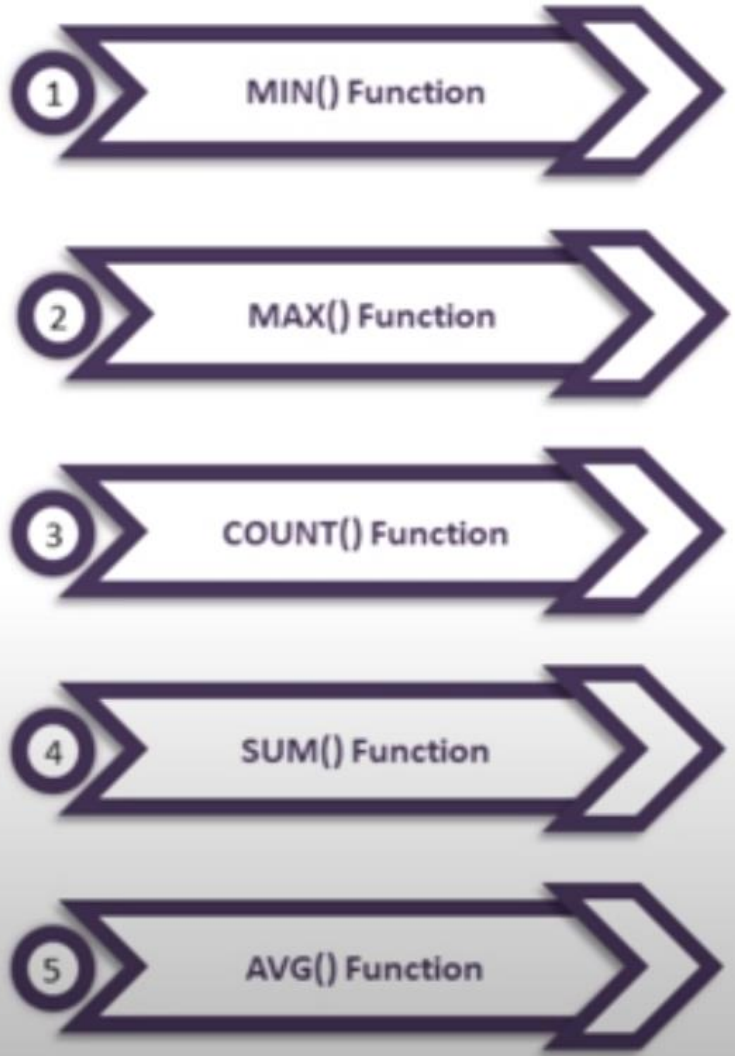



```
SELECT col_list FROM table_name WHERE
column_N LIKE '_XXXX%';
```

BETWEEN Operator Syntax




```
SELECT col_list FROM table_name WHERE
column_N BETWEEN val1 AND val2;
```



e_id	e_name	e_salary	e_age	e_gender	e_dept
1	Sam	95000	45	Male	Operations
2	Bob	80000	21	Male	Support
3	Anne	125000	25	Female	Analytics
4	Julia	73000	30	Female	Analytics
5	Matt	159000	33	Male	Sales
6	Jeff	112000	27	Male	Operations

MIN() Function

MIN() function gives you the smallest value

```
SELECT MIN(col_name) FROM
table_name;
```

MAX() Function

MAX() function gives you the largest value

```
SELECT MAX(col_name) FROM
table_name;
```

COUNT() Function

COUNT() function returns the number of rows that match a specific criteria

```
SELECT COUNT(*) FROM table_name
WHERE condition;
```

SUM() Function

SUM() function gives the total sum of a numeric column

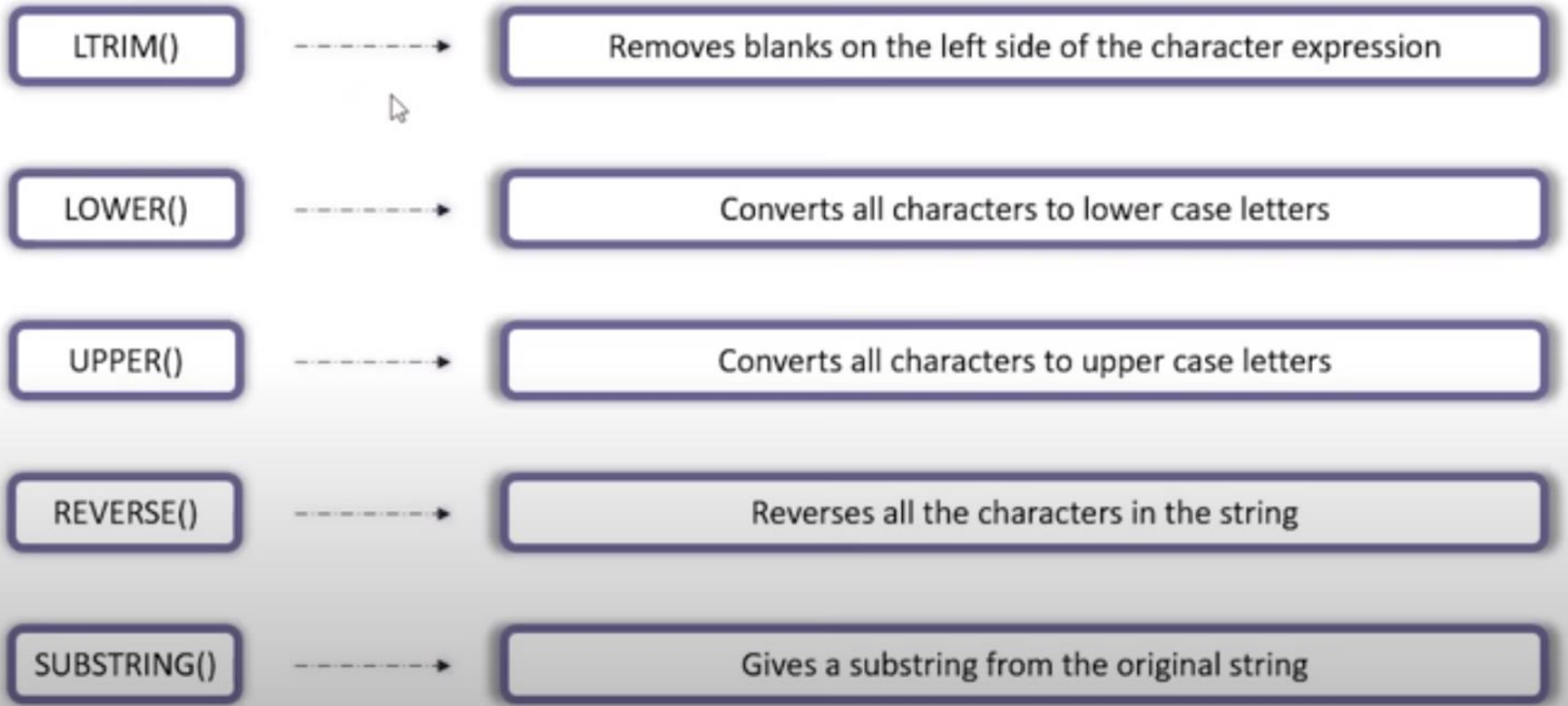
```
SELECT SUM(col_name) FROM
table_name;
```

AVG() Function

AVG() function gives the average value of a numeric column

```
SELECT AVG(col_name) FROM
table_name;
```

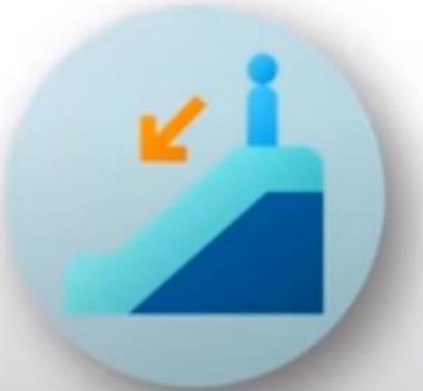
String Functions



ORDER BY

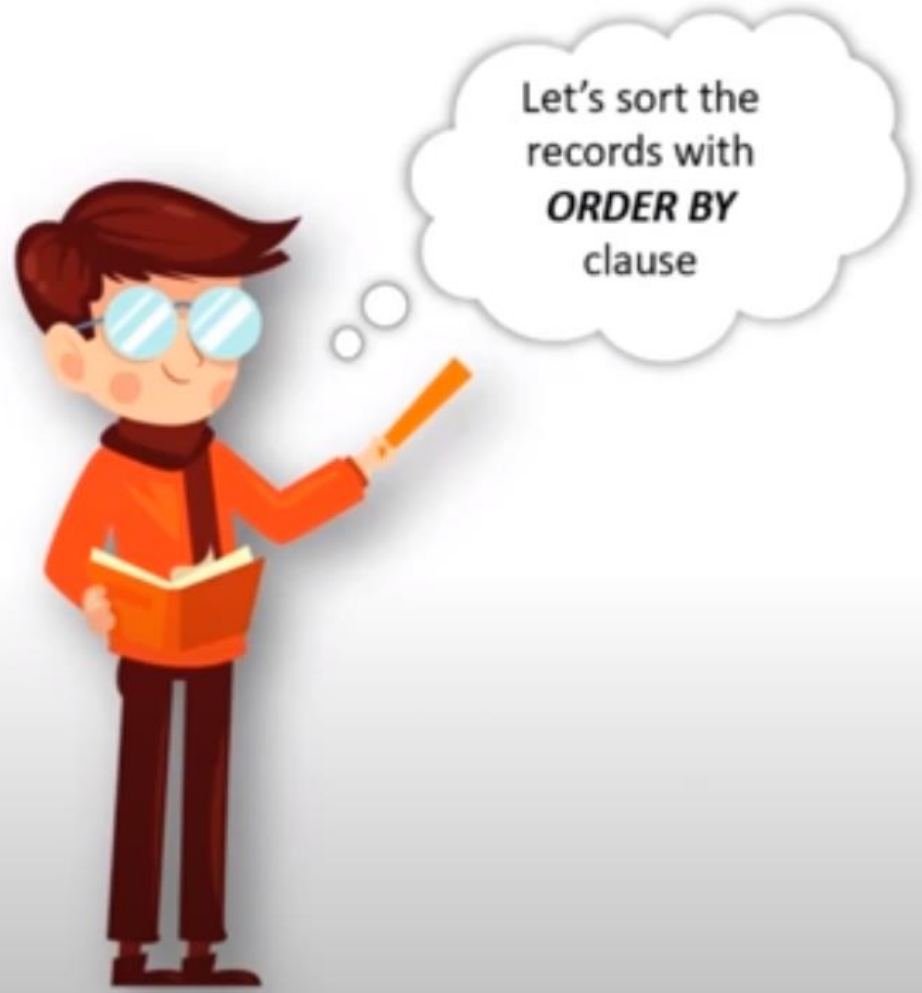
ORDER BY is used to sort the data in ascending or descending order

Ascending



Descending

Order By Syntax




```
SELECT column_list FROM table_name
ORDER BY col1, col2,.....ASC|DESC
```

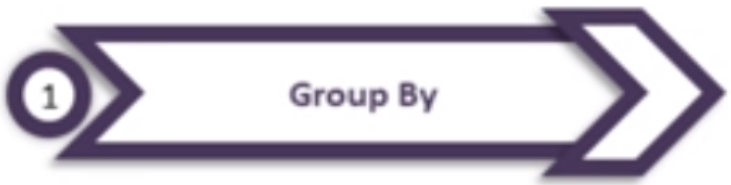
TOP Clause

TOP is used to fetch the TOP N records

e_id	e_name	e_salary	e_age	e_gender	e_dept
1	Sam	95000	45	Male	Operations
2	Bob	80000	21	Male	Support
3	Anne	125000	25	Female	Analytics
4	Julia	73000	30	Female	Analytics
5	Matt	159000	33	Male	Sales
6	Jeff	112000	27	Male	Operations

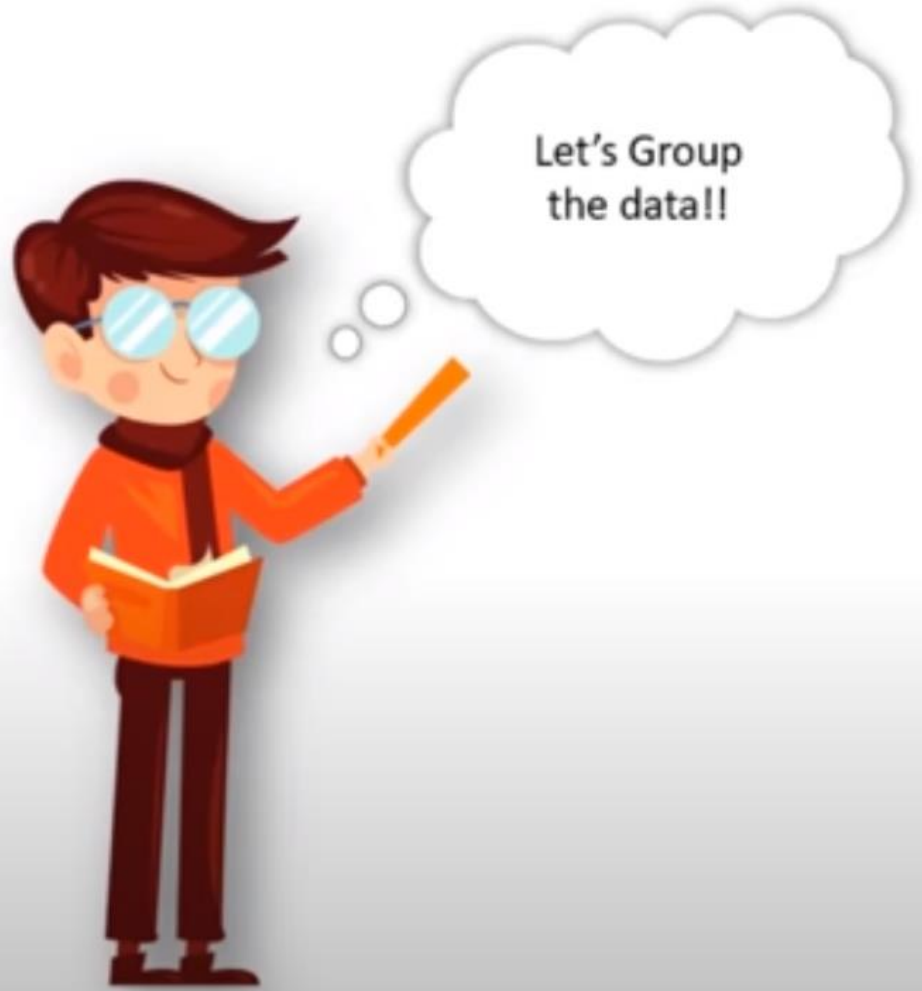
TOP Clause Syntax


```
SELECT TOP x column_list FROM
table_name;
```



e_id	e_name	e_salary	e_age	e_gender	e_dept
1	Sam	95000	45	Male	Operations
2	Bob	80000	21	Male	Support
3	Anne	125000	25	Female	Analytics
4	Julia	73000	30	Female	Analytics
5	Matt	159000	33	Male	Sales
6	Jeff	112000	27	Male	Operations

Group By Syntax

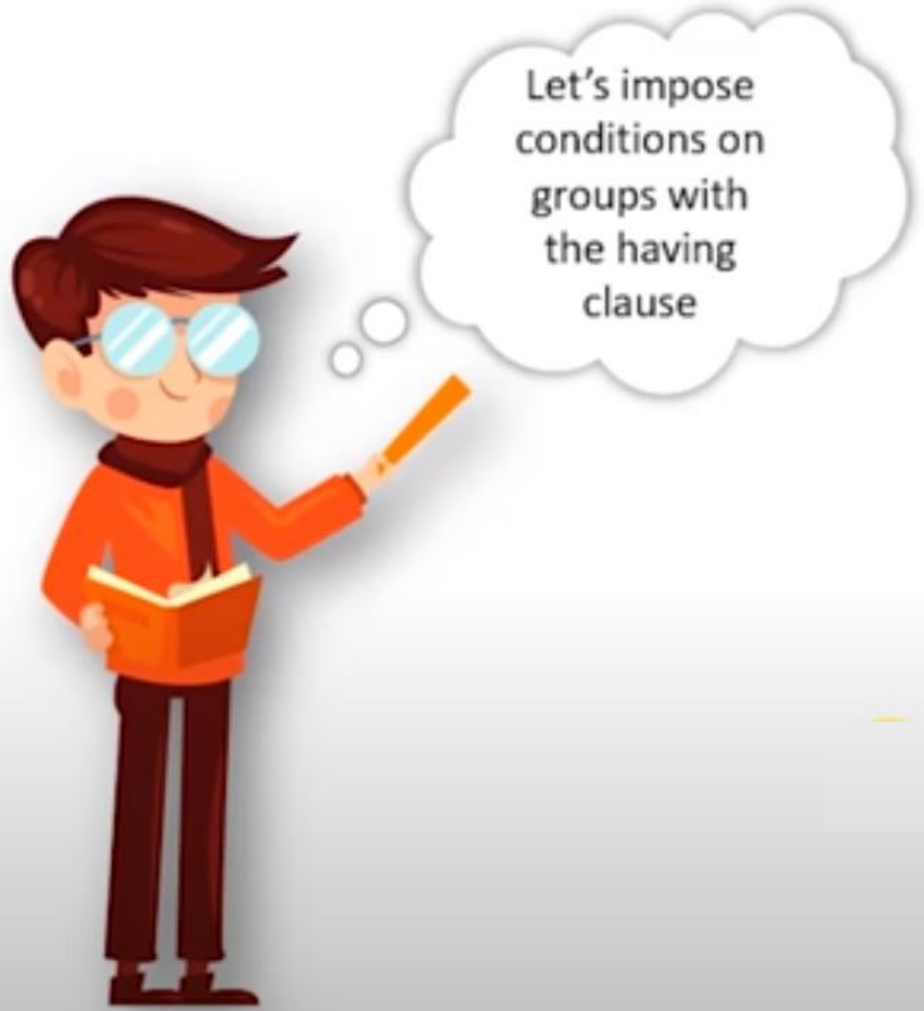



```
SELECT column_list
FROM table_name
WHERE condition
GROUP BY colname(s)
ORDER BY colname(s)
```

Having Clause

Having clause is used in combination with Group By to impose conditions on groups

Having Clause Syntax




```
SELECT column_name(s)
FROM table_name
WHERE condition
GROUP BY column_name(s)
HAVING condition
ORDER BY column_name(s);
```

Update Statement

Update is used to modify the existing records in a table

```
UPDATE table_name
SET col1=val1, col2=val2.....
[WHERE condition];
```

Delete Statement

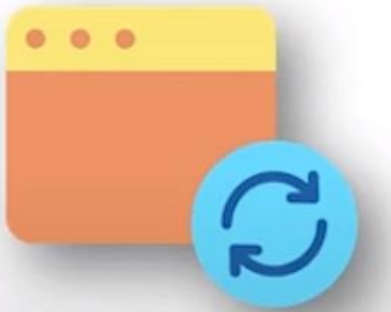
DELETE statement is used to delete existing records in the table

```
DELETE FROM table_name
[WHERE condition];
```

Merge

MERGE is the combination of INSERT, DELETE and UPDATE statements

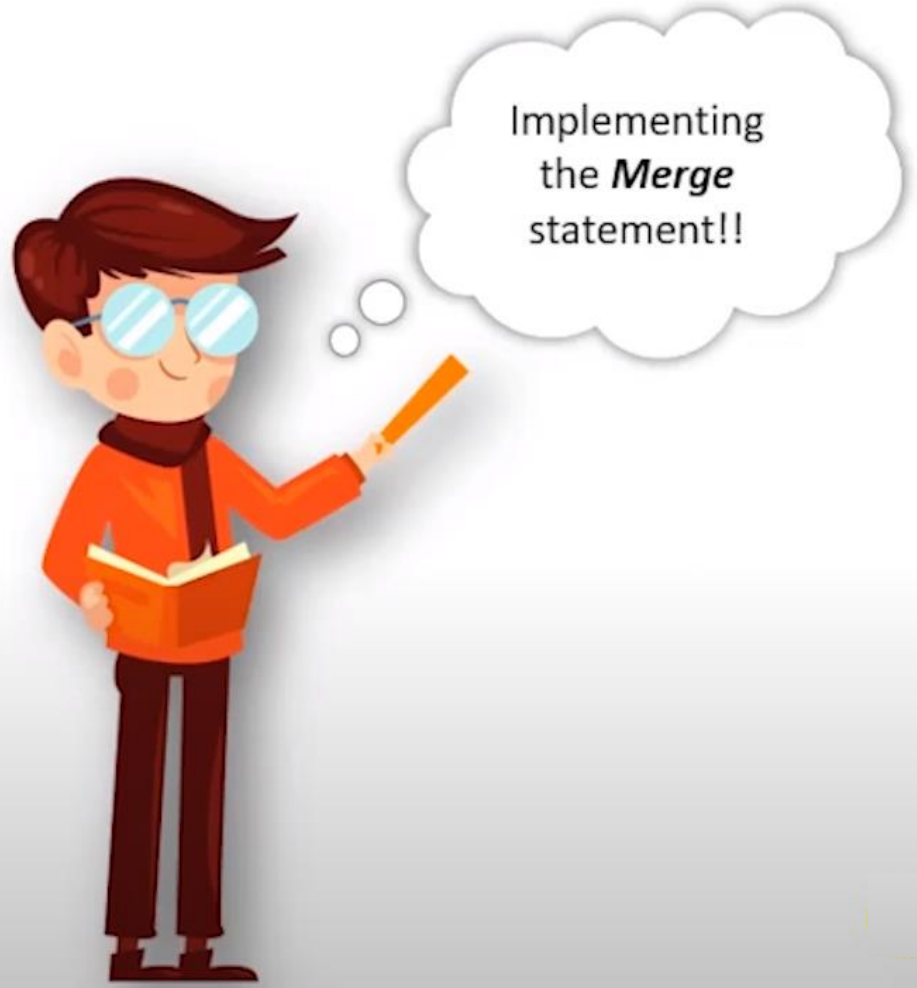
Insert



Update

Delete

Merge




```
MERGE [Target] AS T
USING [Source] AS S
      ON [Join Condition]
WHEN MATCHED
      THEN [Update Statement]
WHEN NOT MATCHED BY TARGET
      THEN [Insert Statement]
WHEN NOT MATCHED BY SOURCE
      THEN [Delete Statement];
```

1 Inner Join in SQL

Employee

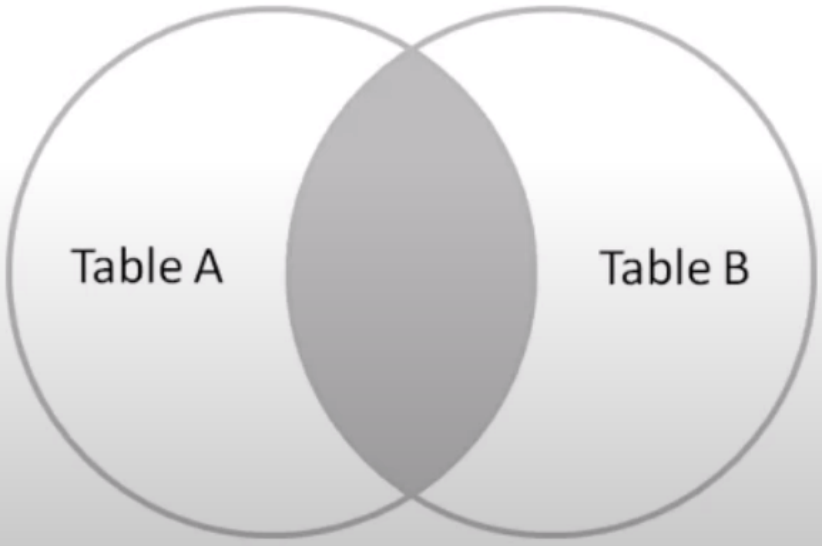
e_id	e_name	e_salary	e_age	e_gender	e_dept
1	Sam	95000	45	Male	Operations
2	Bob	80000	21	Male	Support
3	Anne	125000	25	Female	Analytics
4	Julia	73000	30	Female	Analytics
5	Matt	159000	33	Male	Sales
6	Jeff	112000	27	Male	Operations

Department

d_id	d_name	d_location
1	Content	New York
2	Support	Chicago
3	Analytics	New York
4	Sales	Boston
5	Tech	Dallas
6	Finance	Chicago

Inner Join

Inner Join returns records that have matching values in both the tables. It is also known as simple join

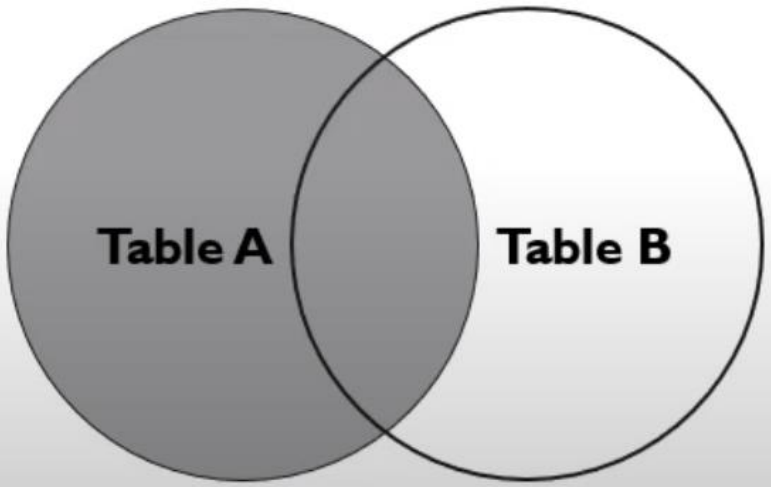


Inner Join : Syntax


```
SELECT columns
FROM table1
INNER JOIN table2
ON table1.column_x = table2.column_y;
```

Left Join

LEFT JOIN returns all the records from the left table, and the matched records from the right table

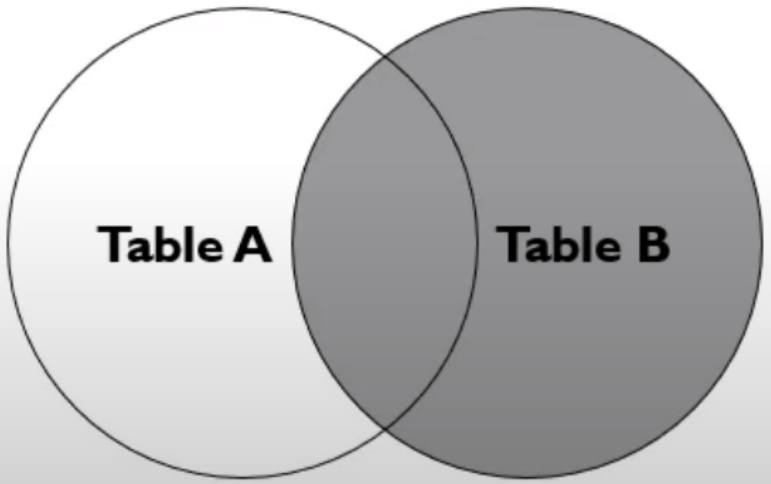


Left Join : Syntax

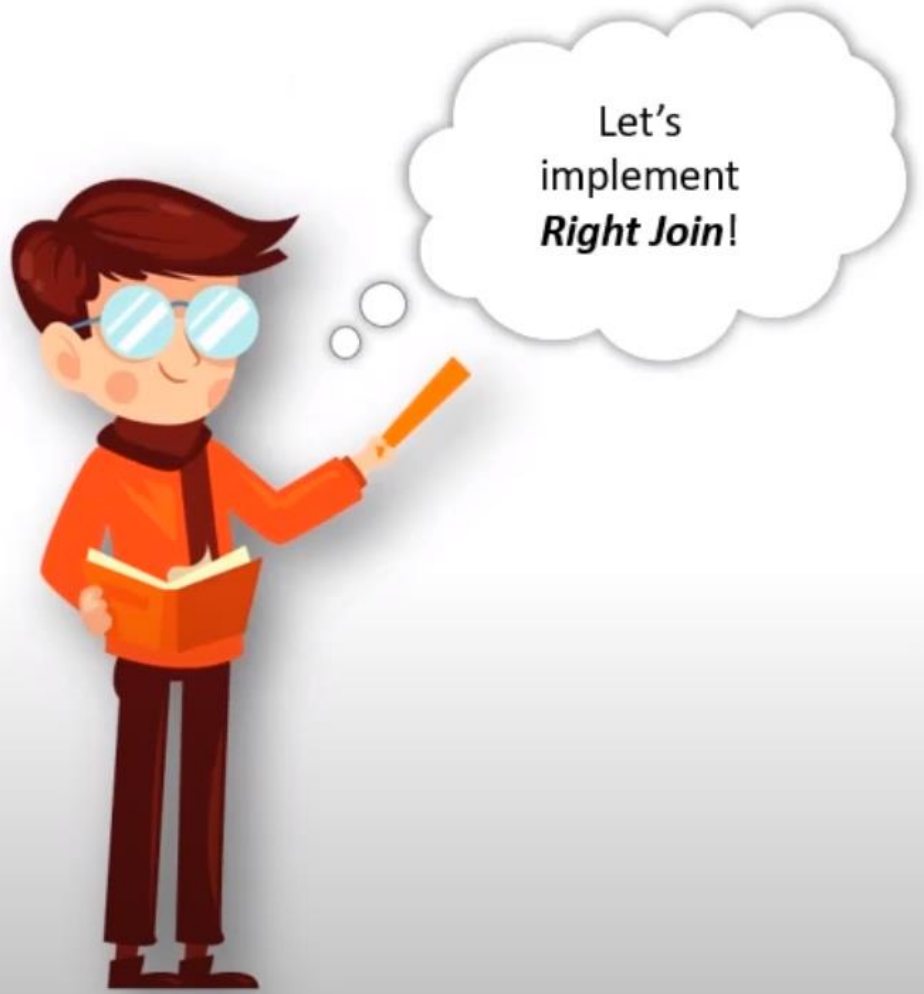

```
SELECT columns
FROM table1
LEFT JOIN table2
ON table1.column_x = table2.column_y;
```

Right Join

RIGHT JOIN returns all the records from the right table, and the matched records from the left table



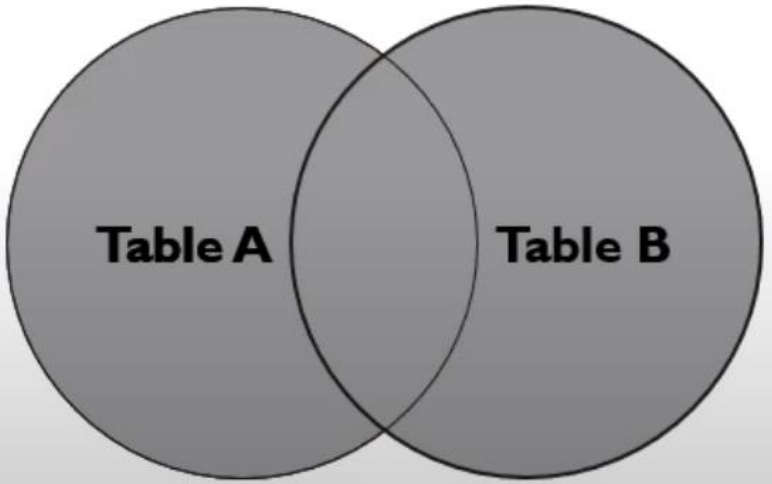
Right Join : Syntax




```
SELECT columns
FROM table1
RIGHT JOIN table2
ON table1.column_x = table2.column_y;
```

Full Join

It returns all rows from the LEFT table and the RIGHT table with NULL values in place where the join condition is not met



Full Join : Syntax


```
SELECT columns
FROM table1
FULL JOIN table2
ON table1.column_x = table2.column_y;
```

Update using Join

Department Table

d_id	d_name	d_location
1	Content	New York
2	Support	Chicago
3	Analytics	New York
4	Sales	Boston
5	Tech	Dallas
6	Finance	Chicago

Employee Table

e_id	e_name	e_salary	e_age	e_gender	e_dept
1	Sam	93000	40	Male	Operations
3	Anne	140000	25	Female	Analytics
6	Jeff	112000	27	Male	Operations
7	Adam	110000	28	Male	Content
8	Priya	85000	37	Female	Tech

Updated Employee Table

e_id	e_name	e_salary	e_age	e_gender	e_dept
1	Sam	93000	40	Male	Operations
3	Anne	140000	35	Female	Analytics
6	Jeff	112000	27	Male	Operations
7	Adam	110000	38	Male	Content
8	Priya	85000	37	Female	Tech

Delete using Join

Department Table

d_id	d_name	d_location
1	Content	New York
2	Support	Chicago
3	Analytics	New York
4	Sales	Boston
5	Tech	Dallas
6	Finance	Chicago

Employee Table

e_id	e_name	e_salary	e_age	e_gender	e_dept
1	Sam	93000	40	Male	Operations
3	Anne	140000	25	Female	Analytics
6	Jeff	112000	27	Male	Operations
7	Adam	110000	28	Male	Content
8	Priya	85000	37	Female	Tech

Modified Employee Table

e_id	e_name	e_salary	e_age	e_gender	e_dept
1	Sam	93000	40	Male	Operations
6	Jeff	112000	27	Male	Operations
8	Priya	85000	37	Female	Tech

Union Operator

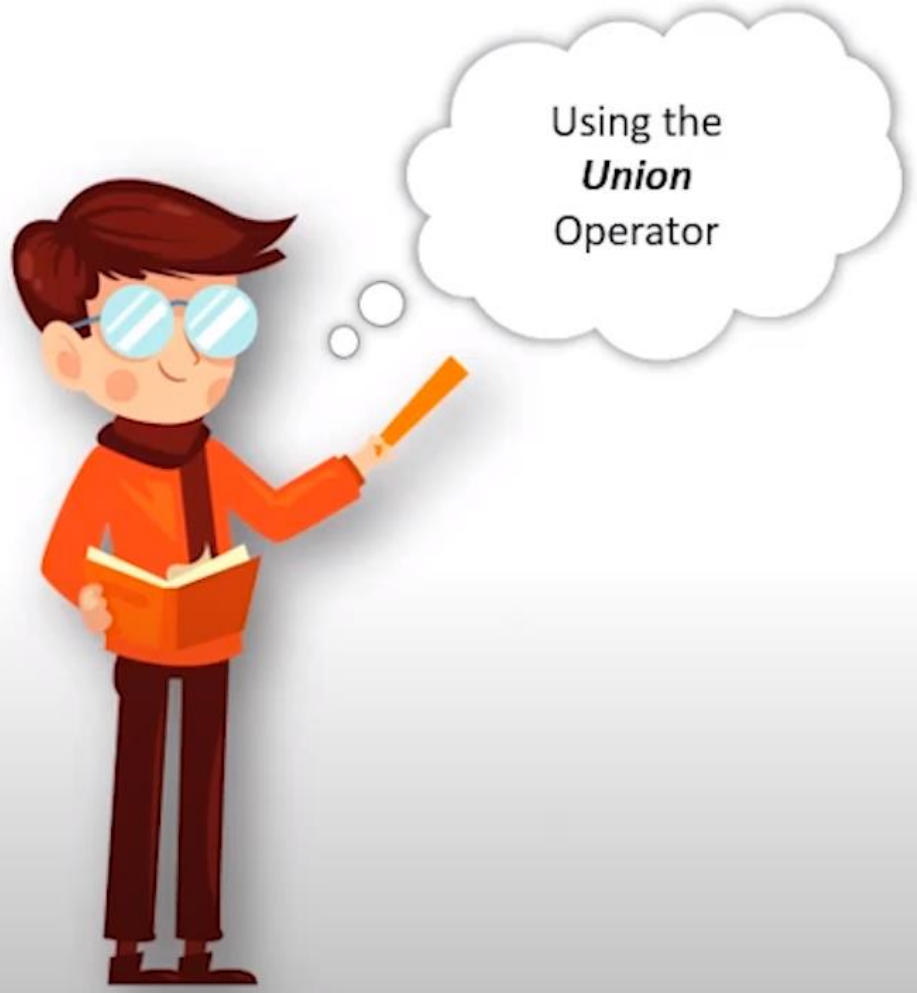
UNION operator is used to combine the result-set of two or more SELECT statements

A

B

A ∪ B

Union Operator Syntax




```
SELECT column_list FROM table1
Union
SELECT column_list FROM table2
```

Union All Operator

UNION All operator gives all the rows from both the tables including the duplicates

A

B

A union all B

Except Operator

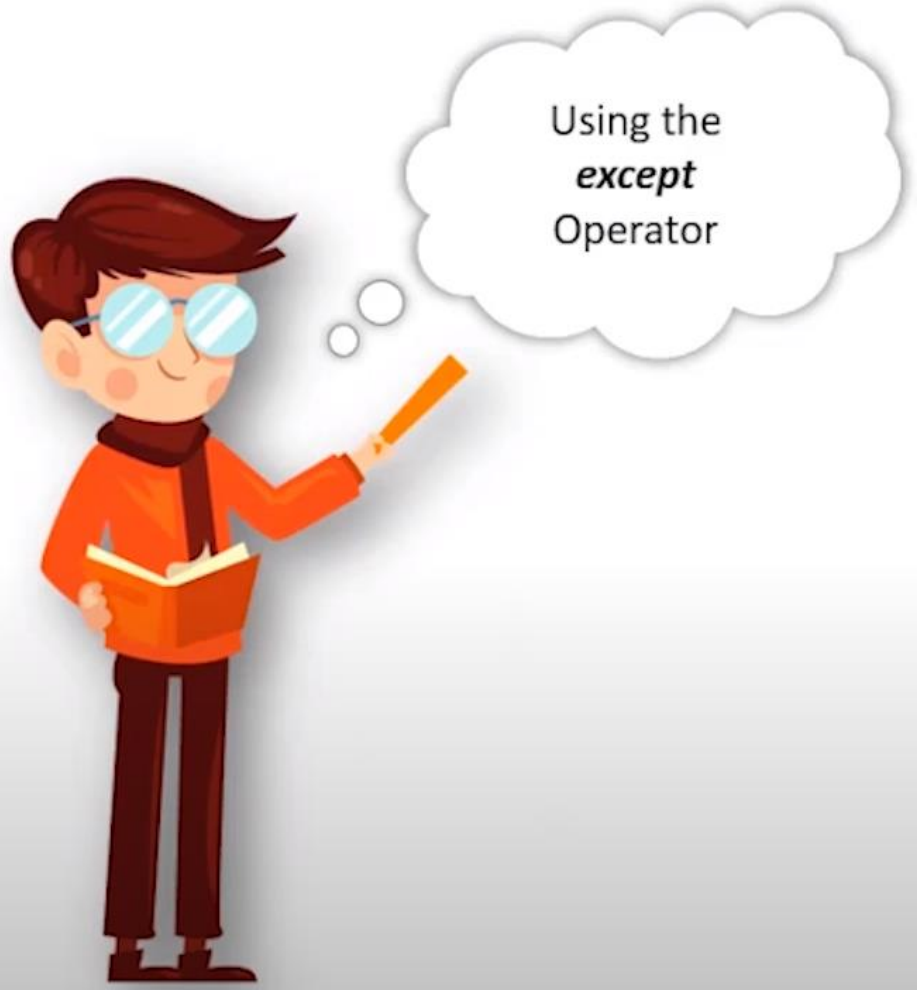
Except Operator combines two select statements and returns unique records from the left query which are not part of the right query

A

B

A - B

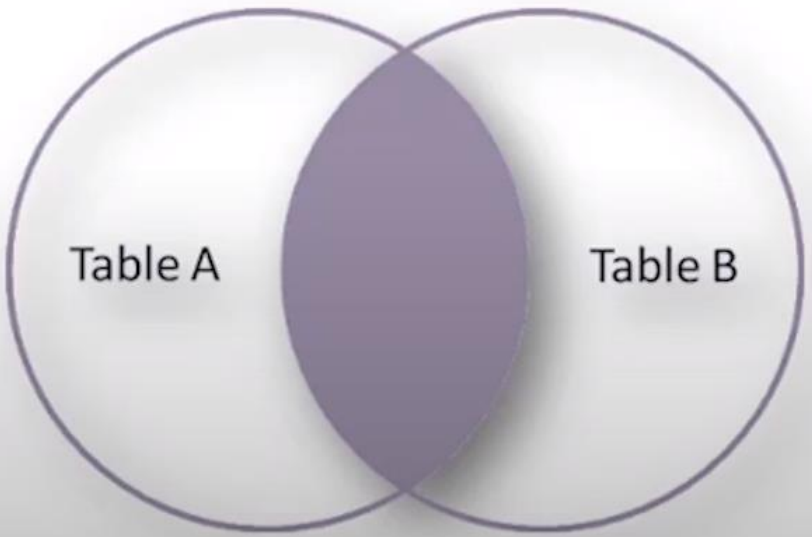
Except Operator Syntax




```
SELECT column_list FROM table1
EXCEPT
SELECT column_list FROM table2
```

Intersect Operator

Intersect Operator helps to combine two select statements and returns the records which are common to both the select statements



Intersect Operator Syntax


```
SELECT column_list FROM table1
INTERSECT
SELECT column_list FROM table2
```

Views

View is a virtual table based on the result of an sql statement

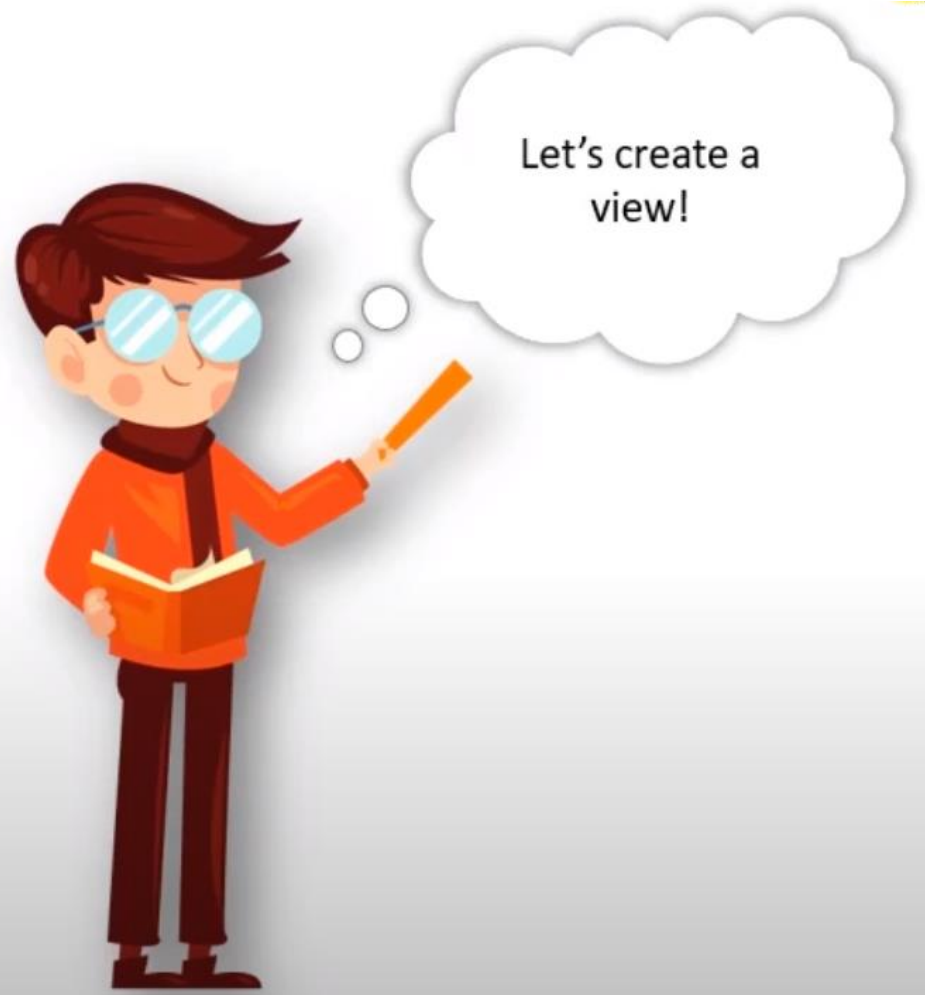
e_id	e_name	e_salary	e_age	e_gender	e_dept
1	Sam	95000	45	Male	Operations
2	Bob	80000	21	Male	Support
3	Anne	125000	25	Female	Analytics
4	Julia	73000	30	Female	Analytics
5	Matt	159000	33	Male	Sales
6	Jeff	112000	27	Male	Operations

Employee Table

e_id	e_name	e_salary	e_age	e_gender	e_dept
3	Anne	125000	25	Female	Analytics
4	Julia	73000	30	Female	Analytics

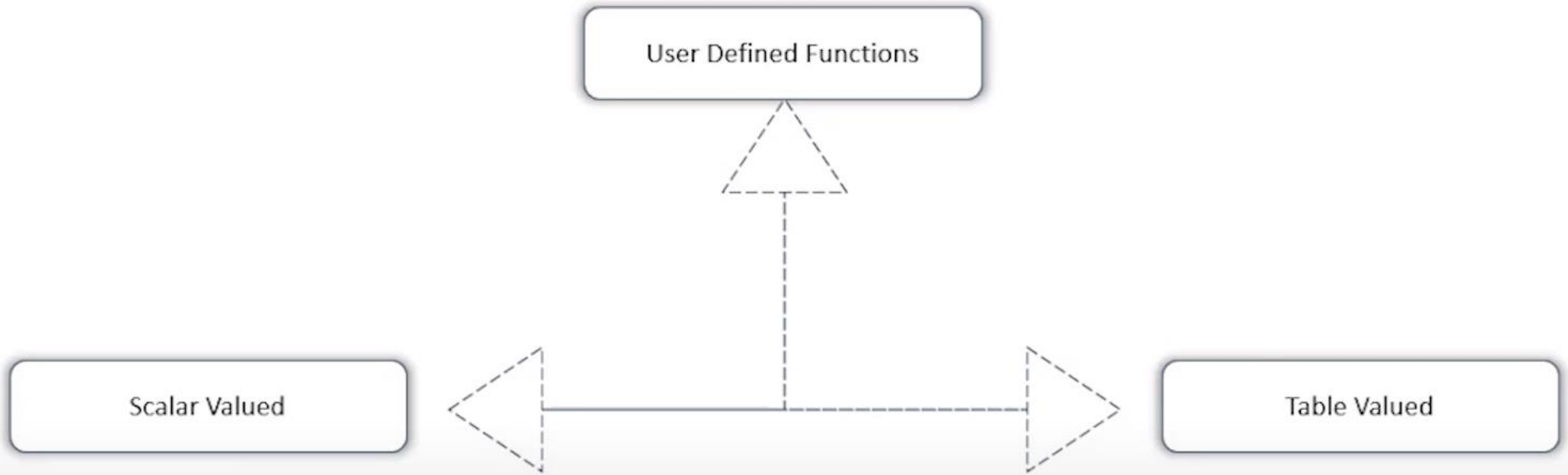
"Female_Employee" View

Create View Syntax




```
CREATE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

Types of User Defined Functions



Scalar Valued Function

Scalar valued function always returns a scalar value

```
CREATE FUNCTION function_name(@param1 data_type, @param2 data_type…)
RETURNS return_datatype
AS
BEGIN
      -----Function body
RETURN value
 END
```

Table Valued Function

Table valued function returns a table

```
CREATE FUNCTION function_name(@param1 data_type, @param2 data_type…)
RETURNS table
AS
RETURN (SELECT column_list FROM table_name WHERE [condition] )
```

Temporary Table

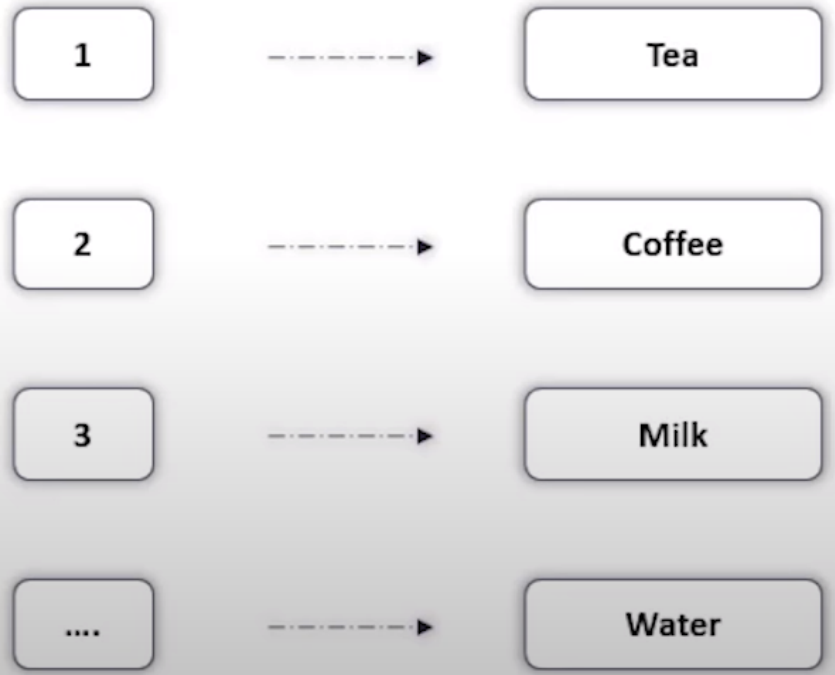
Temporary tables are created in tempDB and deleted as soon as the session is terminated

Syntax to Create Temporary Table


```
CREATE TABLE #table_name(
);
```

Case Statement

Case Statement helps in multi way decision making



Case Statement Syntax


```
CASE
   WHEN condition1 THEN result1
   WHEN condition2 THEN result2
   WHEN conditionN THEN resultN
   ELSE result
END;
```

IIF() Function

IIF() function is an alternative for the case statement

```
IIF (boolean_expression, true_value, false_value )
```

Stored Procedure in sql

STORED PROCEDURE is a prepared sql code which can be saved and reused

Stored Procedure without parameter Syntax


```
CREATE PROCEDURE procedure_name
AS
sql_statement
GO;
```

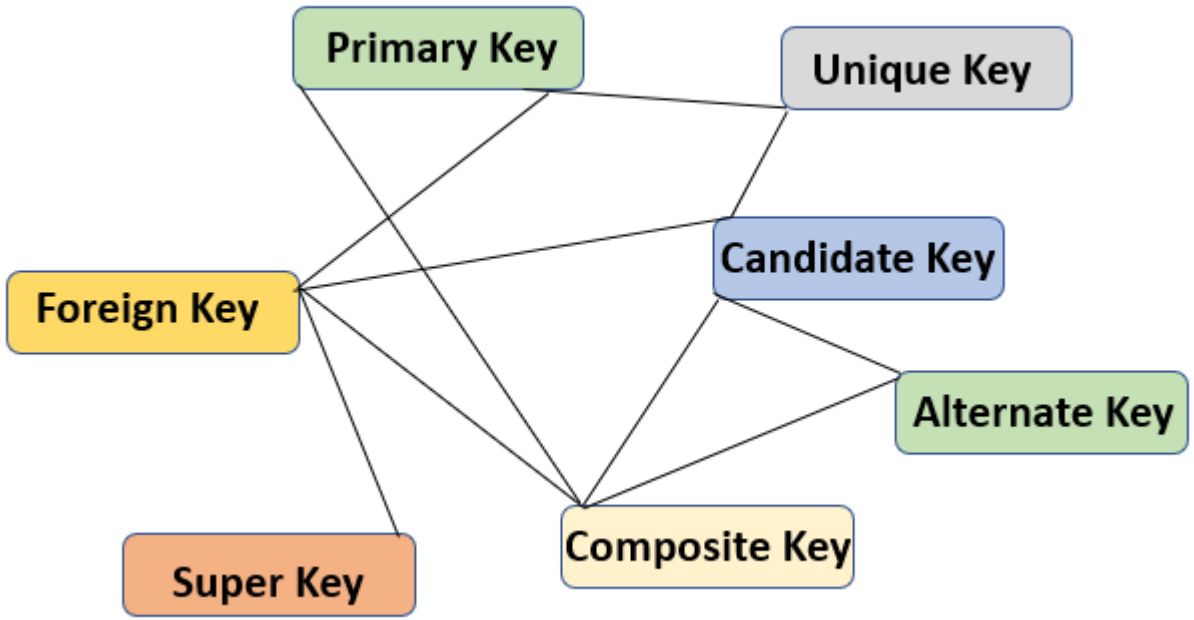
↓

```
EXEC procedure_name
```

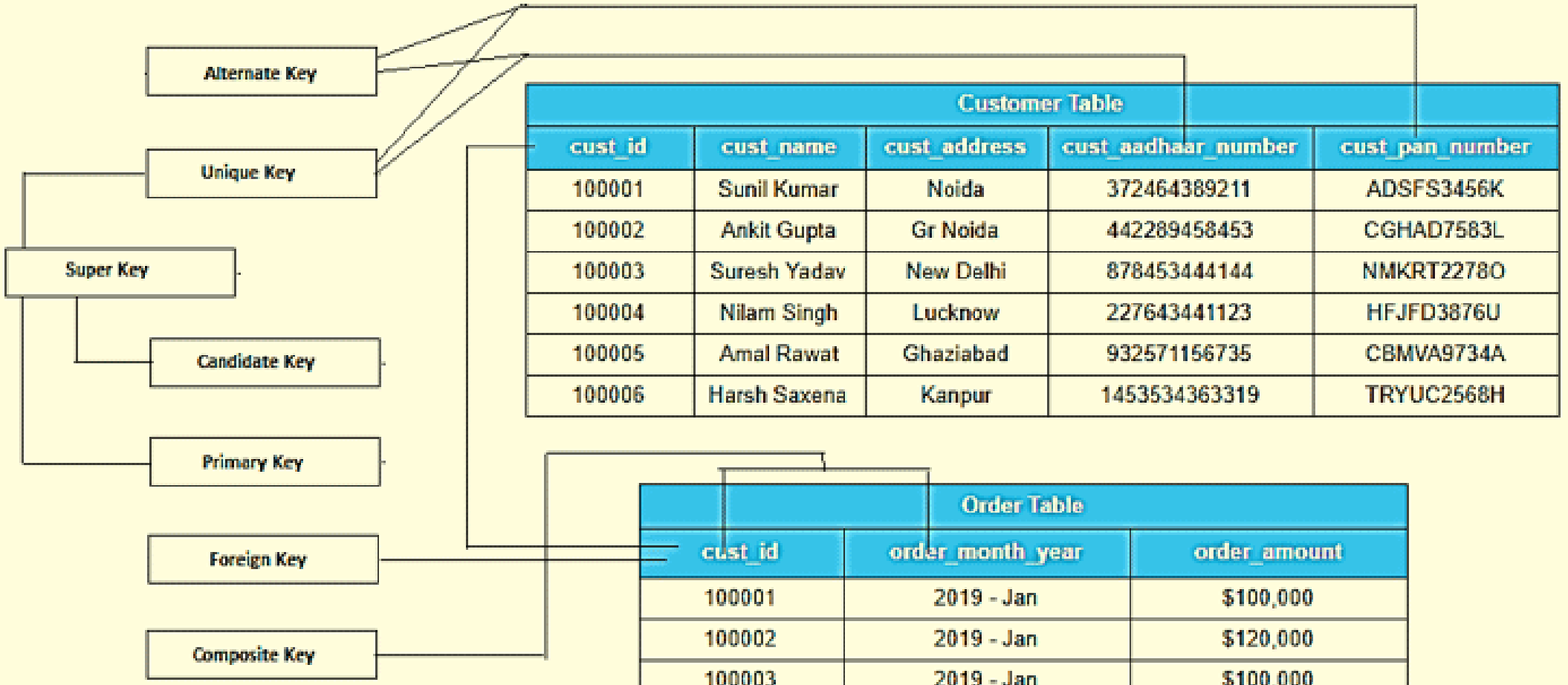
Stored Procedure with parameter Syntax


```
CREATE PROCEDURE procedure_name
@param1 data-type, @param2 data-type
AS
sql_statement
GO;
```

Type of SQL Keys



Type of SQL Keys



Customer Table

cust_id	cust_name	cust_address	cust_aadhaar_number	cust_pan_number
100001	Sunil Kumar	Noida	372464389211	ADSFS3456K
100002	Ankit Gupta	Gr Noida	442289458453	CGHAD7583L
100003	Suresh Yadav	New Delhi	878453444144	NMKRT2278O
100004	Nilam Singh	Lucknow	227643441123	HFJFD3876U
100005	Amal Rawat	Ghaziabad	932571156735	CBMVA9734A
100006	Harsh Saxena	Kanpur	1453534363319	TRYUC2568H

Order Table

cust_id	order_month_year	order_amount
100001	2019 - Jan	$100,000
100002	2019 - Jan	$120,000
100003	2019 - Jan	$100,000
100004	2019 - Jan	$110,000
100001	2019 - Feb	$105,000
100002	2019 - Feb	$125,000

Exception Handling

Exception Handling

An error condition during a program execution is called an exception

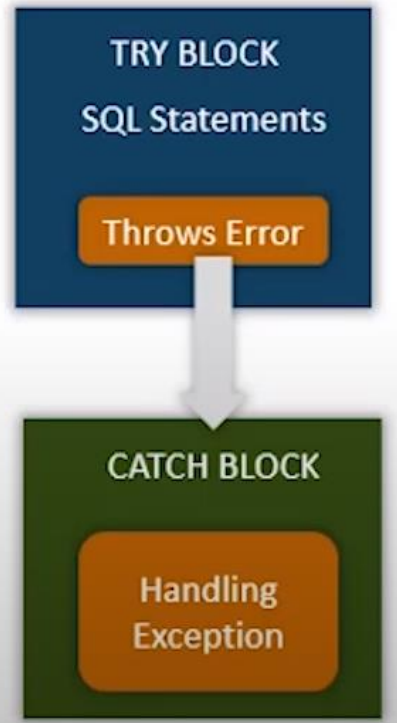
Exception

The mechanism for resolving such an exception is exception handling

Exception Handling

Try/Catch

SQL Provides the try/catch blocks for exception handling



Try/Catch Syntax


```
BEGIN TRY

SQL Statements

END TRY

BEGIN CATCH

- Print Error OR
- Rollback Transaction

END CATCH
```

Transactions in SQL

Transaction is a group of commands that change data stored in a database

```
begin try
   begin transaction
            update employee set e_salary=50
where e_gender='Male'
            update employee set
e_salary=195/0 where e_name='Female'
            commit transaction
   Print 'transaction committed'
end try
begin catch
   rollback transaction
            print 'transaction rolledback'
end catch
```

Transactions in SQL

Transaction is a group of commands that change data stored in a database

```
begin try
   begin transaction
            update employee set e_salary=50
where e_gender='Male'
            update employee set
e_salary=195/0 where e_name='Female'
            commit transaction
   Print 'transaction committed'
end try
begin catch
   rollback transaction
            print 'transaction rolledback'
end catch
```

Transactions in SQL

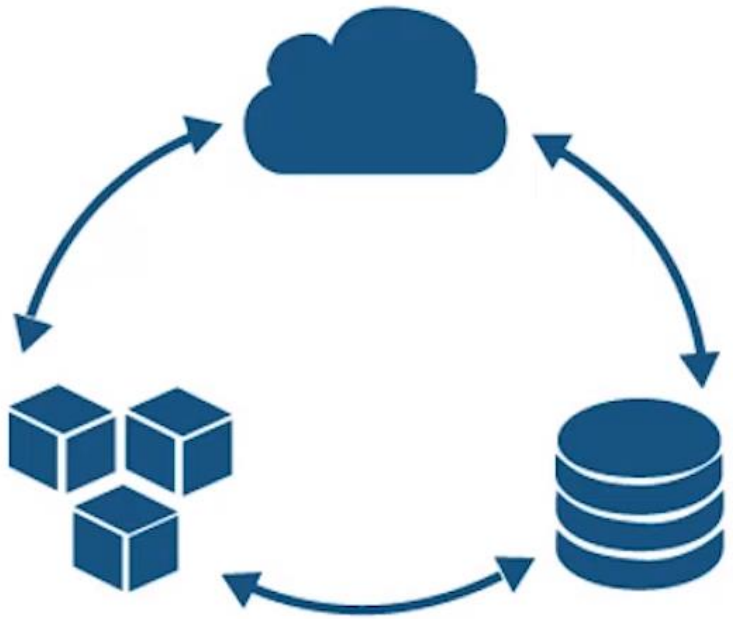
Transaction is a group of commands that change data stored in a database

```
begin try
   begin transaction
            update employee set
e_salary=50 where e_gender='Male'
            update employee set
e_salary=195 where e_name='Female'
            commit transaction
   Print 'transaction committed'
end try
begin catch
   rollback transaction
            print 'transaction rolledback'
end catch
```

More on:

https://www.sqlservercentral.com/articles/sql-and-t-sql-for-beginners-and-totally-beginners-in-229-minutes

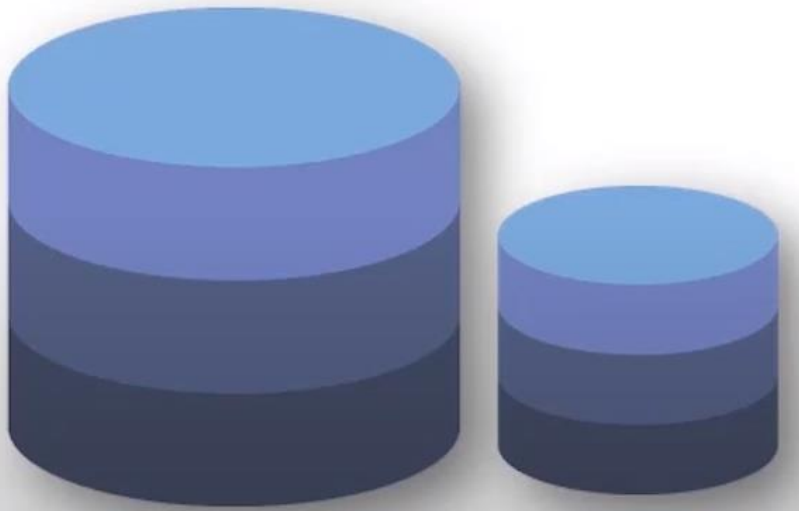
https://o7planning.org/en/10327/sql-server-transact-sql-programming-tutorial



Microsoft SQL Server DBA

Data Base Administrator

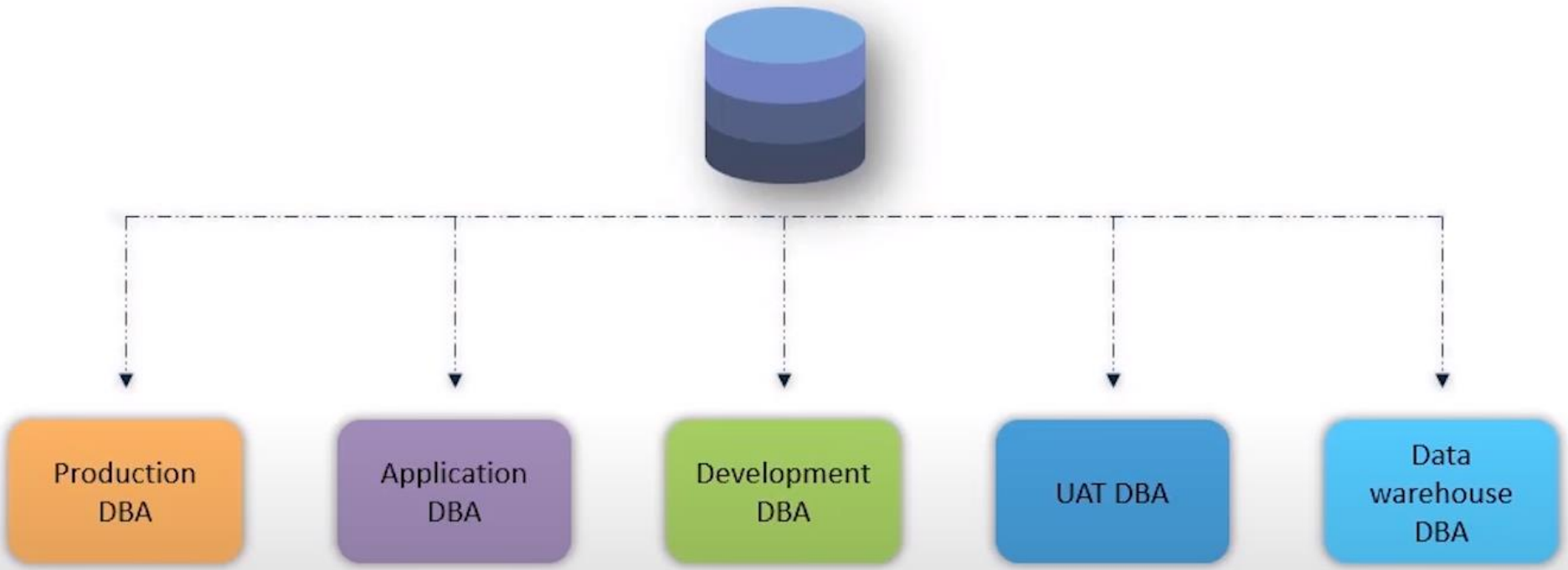
A database administrator (DBA) directs or performs all activities related to maintaining a successful database environment



Roles of Data Base Administrator

1. Software Installation and Maintenance
2. Taking care of Database backup & recovery
3. Maintaining security of the database
4. Take care of Access Control for different users
5. Monitor databases for performance issues

Types of DBA


Production DBA
Application DBA
Development DBA
UAT DBA
Data warehouse DBA